

RESEARCH REPORT

Generalized Transformers**Geometry, Compression, Predictive Coding,
and Continual Learning**

An architecture for learning systems that keep learning: the framework,
a working prototype, first experimental evidence, and a plan for scaling up

Prepared for Ben Goertzel and colleagues, with ChatGPT assistance

September 18, 2026

Abstract

We would like learning systems that keep acquiring knowledge while they are in use without losing what they already know, that can hand a newly learned capability to another system as a small module that can be checked before it is trusted, that connect naturally to symbolic reasoning, and that can build a new representation when the familiar ones fail. Transformers are a strong foundation, but fitting the weights of a fixed architecture does not supply these properties by itself. This report describes a research program aimed at them, and what its first experiments show.

The program has four parts. The first is a broader description of what a transformer layer does, which we call a *generalized transformer* (GT). A GT layer decides where to look, brings back evidence, makes useful relationships explicit, reconciles competing evidence when necessary, and keeps a representation suited to whatever comes next. The description is geometric: the questions one can ask of a state define distances between states, routing is a set of input-dependent probability weights, and compression is judged by what it does to later behavior. An ordinary transformer block is one special case. The second part, *compression-driven distinction* (CD) networks, is another. In a CD network the representation is an executable structure of reusable features together with the exceptions those features fail to explain, and a bounded search, guided by a record of earlier searches, proposes and tests changes to that structure. The third part concerns how the numerical parameters inside such a structure are learned: by ordinary backpropagation (BP), or by predictive coding (PC), a family of local, energy-based procedures. The fourth part is a practical route to experiments. The same mechanisms can be attached as a *cap* on top of a pretrained transformer, so that the new machinery only has to improve on a model that already works.

Each idea is presented first in plain language and then mathematically, with worked examples and algorithms. We then report a series of small experiments. The strongest cap built so far improves next-character prediction on the Tiny Shakespeare corpus substantially, mainly because it gives the model a richer memory to read from; structural selection and prediction-time inference add smaller, consistent gains. PC learning does not outperform a matched BP control. When the cap is adapted to a new domain it forgets a moderate and consistent amount of its earlier skill, and a small replay buffer removes a good part of that loss. These results support particular mechanisms and a practical research path. They leave open decentralized operation, broad creative generalization, and any advantage at frontier scale, and the final part of the report lays out a staged plan, with explicit decision criteria, for investigating those questions.

How to read this report. No earlier document on this project is assumed. Section 1 explains the goals and gives a short overview, and Section 2 reviews the standard ideas the rest depends on. Every technical section opens with an informal account of its idea before any notation appears, and the running example of Section 3.2 recurs throughout, so a reader who wants the concepts without the mathematics can follow those openings and the discussion after each equation. What was built and measured is in Sections 7–9, and what comes next in Section 10. The appendices hold proofs, exact bounds, full tables, implementation details and the provenance of every number, and a glossary is at the end.

Contents

1	Introduction: why look beyond the standard transformer?	4
1.1	The starting point	4
1.2	Five goals, and what would count as progress	4
1.3	The approach in brief	5
1.4	What has been tested	6
1.5	Relation to existing work	7
2	Background in brief	7
3	The generalized transformer	8
3.1	The idea: six jobs in one layer	8
3.2	A running example	9
3.3	Observers and the geometry of distinctions	9
3.4	Routing: an input-dependent way of sharing out attention	10
3.5	Transport, aggregation, lifting and re-encoding	11
3.6	The ordinary transformer is an exact special case	12
3.7	Why the order of operations can decide what survives	12
4	Compression-driven distinction networks	13
4.1	The idea: rules, exceptions, and a better index	13
4.2	What a compressed state must preserve	14
4.3	Executable feature-and-residual representations	15
4.4	When is a new module worth keeping?	16
4.5	Bounded discovery, reuse and adaptation	16
4.6	Dependency towers and a search guide that learns from history	17
4.7	Why higher-order observations are sometimes indispensable	18
4.8	Training the guide: warm-up, label quality, and richer features	18
5	Caps: attaching CD to an existing transformer	19
5.1	The idea	19
5.2	Two ways for a cap to write its output	20
5.3	Composable modules need to be trained together	20
5.4	Distilling a base so that caps fit it well	21
6	Learning inside the structure: predictive coding and backpropagation	22
6.1	The idea: revise the answer, the habit, or the machinery	22
6.2	An energy-based realization of the GT layer	22
6.3	How prediction errors can inform structural search	23
6.4	Three timescales, and learning while serving	23
7	The working cap: a concrete generalized-transformer instance	24
7.1	Readers that report full distributions	24
7.2	The gate: a learned prior adjusted by recent evidence	25
7.3	Choosing the structure: which readers to keep	26
7.4	H2 as a generalized transformer	27
8	Experimental evidence	27
8.1	How to read these experiments	27
8.2	Finite program-discovery tasks	27
8.3	Early text prototypes	28
8.4	A revised admission criterion, and further development	28
8.5	CD text v6: useful mechanisms, uneven outcomes	29
8.6	Hybrid H1: a useful negative result	31
8.7	Hybrid H2: richer readers, explicit inference, and compression	31

8.8	Does settling at prediction time add anything?	32
8.9	What the experiments support so far	33
9	Forgetting: what happens when the cap learns something new?	33
9.1	Protocol, and what is meant by forgetting	33
9.2	Results: consistent interference, with most capability retained	34
9.3	Why frozen readers can coexist with forgetting	36
9.4	What can be guaranteed about retention	36
9.5	The continual-learning question that remains	37
10	Scaling from the sandbox toward frontier systems	37
10.1	What survives approximation, and what does not	37
10.2	Stages of data and compute, with decision criteria	37
10.3	A budget equation	39
10.4	The vocabulary and reader bottleneck	40
10.5	Predictive coding at scale: keep the useful inference, and charge for the settling	40
10.6	Decentralization as the exchange of modules	41
10.7	Linking to symbolic systems, and testing creativity	41
10.8	What would justify a frontier-scale claim?	42
11	Assessment and research priorities	42
A	Geometric preservation and the composition of errors	42
A.1	Derivation of the local predictive metric	43
A.2	Preserving a composition of stages	43
B	Exact bounds for finite compression decisions	43
B.1	A compatible histogram and a cost table	44
B.2	Checking numerical LP output conservatively	45
B.3	Sparse dependencies and a bound on the remainder	45
B.4	Partial evaluation and sampled evaluation	45
B.5	A useful convex subproblem: mixture weights	46
C	A finite compression example with exact arithmetic	46
D	Implementation and data details	47
D.1	The H2 reader grammar and fitting configuration	47
D.2	Forgetting data and replay	48
E	Additional numerical tables	48
E.1	The full H2 comparison	48
E.2	The earlier v6 evaluation segment	48
E.3	Forgetting ranges and validation-selected endpoints	50
F	Provenance of the evidence, verification and reproduction	50
F.1	Where the numbers come from	50
F.2	The scope of the recorded validation	51
F.3	A suggested sequence for replication	51
G	Glossary	52

1 Introduction: why look beyond the standard transformer?

1.1 The starting point

A present-day language model is built in two separate phases. During training, a very large number of numerical weights are adjusted so that the model predicts text well. During use, those weights are frozen: the model reads a prompt and answers, and nothing it encounters changes what it knows the next day. This separation is a large part of why the technology works so well, and it is also the source of several limitations that become visible as soon as one imagines a system that develops through its own experience.

A capable conversational model needs a separate training run to turn a useful experience into lasting knowledge. When its weights are updated on new material, its earlier behavior can deteriorate, sometimes badly. Passing a newly acquired skill from one model to another usually means running a substantial adaptation pipeline, because the skill is spread across millions of weights and cannot simply be copied out. Facts held in a symbolic reasoner are hard to connect to the particular internal computation that needs them. And a model may produce fluent variations on what it has seen without reliably discovering a new computational abstraction that would solve an unfamiliar kind of problem.

None of these is an impossibility result about transformers. A transformer can be trained online, given an external memory, equipped with tools, or embedded in a decentralized system, and existing work on continual learning, test-time training, retrieval, and parameter-efficient adaptation addresses parts of the problem [2, 3, 12, 11]. The question pursued here is narrower: whether an architecture that is explicit about what information it preserves, how new modules are discovered, and how adaptation is kept local makes these capabilities easier to combine and easier to measure.

1.2 Five goals, and what would count as progress

Five goals motivate the work. For each we say what we mean, and what would and would not count as achieving it, because several of them are easy to satisfy in a trivial way.

Continual learning with controlled forgetting. The system should learn a new domain while keeping its useful old behavior. A frozen model does not meet this goal, since it keeps its old skill only by learning nothing. A lossless archive of everything ever seen does not meet it either if the system cannot find the relevant evidence when it is needed. We therefore keep three things apart: what knowledge is stored, whether the system can get at it, and how well it performs when it uses it. The forgetting experiment in Section 9 isolates a failure of the second kind. The stored knowledge stays fixed, and what deteriorates is the learned policy for consulting it.

Learning during use. Prediction, short-lived adjustment of internal state, parameter updates, and the discovery of new structure should be able to coexist, each on its own timescale. An agent should be able to go on giving useful answers while a slower process investigates an error that keeps recurring. This does not require every parameter to change during every forward pass. It requires serving and learning to be parts of one versioned process, with a well-defined order of events and a bound on how much a running prediction can be disturbed. Predictive coding offers a natural vocabulary for the adjustment of internal state, but concurrency also depends on scheduling and on how versions and memories are stored.

Decentralized acquisition and exchange of capabilities. Different learners see different data and have different amounts of compute. We would like them to exchange small executable modules, summaries of

Goal	Architectural route	Evidence currently available
Retention	Protected modules, replay, stable interfaces, retention-aware routing	Moderate forgetting measured; small replay buffer helps
Learning during use	Fast prediction plus slower local updates and discovery	Prediction-time inference and adaptation both run; concurrent serving is a design
Decentralized learning	Versioned modules and portable evidence; local acceptance	No distributed benchmark yet
Neural-symbolic linkage	Typed readers, executable features, provenance	Finite observer/program grammars work; no full Hyperon integration
Creative generalization	Search over compositions, dependency scopes and representations	Some finite motifs and compositions recovered; broad novelty untested

Table 1: The five goals, the route proposed toward each, and the evidence so far.

evidence, or records of search experience, without always rebuilding a single global model. Federated learning is a precedent for learning from distributed data [4], although in its usual form a central server aggregates the updates, which is different from the peer-to-peer ecosystem of modules envisaged here. A decentralized design has to cope with data that differ from site to site, interfaces that go out of date, modules that conflict, the cost of communication, and the need for each recipient to test a module before accepting it. Local learning equations do not solve those problems by themselves.

Closer links between neural and symbolic computation. A reasoning engine, knowledge graph, program interpreter or planner should be able to supply a typed observation to the same workspace that a neural component reads. In the other direction, a distinction the system has learned should, where possible, be exportable as an executable module that a person or program can inspect. For systems such as Hyperon, MeTTa, PLN and Omega, an attractive interface is a module with declared inputs and outputs, a statement of uncertainty, a record of where its evidence came from, and a cost. This is an architectural proposal. The experiments reported here do not yet integrate those systems.

More useful creativity outside the training distribution. When familiar computations fail, the system should be able to construct and test a new composition or a new representation. A short program that combines known operators in a way not seen before is one concrete target. Novelty alone is not the aim: a creative candidate has to help on unseen instances and survive factual or executable checks. A fixed, finite grammar of operators can be used to test the discovery of new compositions within that grammar, though it cannot show the invention of entirely new primitives. Benchmarks of compositional generalization motivate this distinction [5].

Table 1 sets each goal beside the architectural route proposed for it and the evidence available today. The goals are considerably broader than the present empirical claims, and the table is meant to keep that visible.

1.3 The approach in brief

The report develops four ideas, and they are easier to follow with the whole picture in view.

The first is a general description of what one layer of a transformer-like system does. At each position in a text the layer works out what it needs, chooses where to look, brings back evidence, computes features that make useful relationships easy to see, reconciles conflicting evidence if necessary, and writes down a state for the next layer. We call any layer organized this way a *generalized transformer* (GT) layer. The

standard transformer block fits the description exactly, with particular choices at each step, and so do systems that read from dictionaries, run small programs, or keep explicit records. The description is geometric in a specific sense explained in Section 3: it is built from distances between states, probability weights over sources, and maps that merge states which need not be told apart.

The second idea is a particular kind of GT layer, the *compression-driven distinction* (CD) network. A CD network tries to explain its observations with reusable rules, stores separately the exceptions that the rules do not cover, and keeps an index so that the right pieces can be found quickly. When errors recur it searches for a better explanation, such as a new rule or a new combination of old ones, refits the numerical parts that are affected, and keeps the change if it helps. A record of past searches guides later ones. The name reflects the two principles at work: good explanations compress the data, and what a representation must preserve are the distinctions between situations that later computations depend on.

The third idea concerns how the numerical parameters inside such a structure are fitted. *Backpropagation* (BP) is the standard method. *Predictive coding* (PC) is an alternative in which the network first lets its internal activities settle into a state that balances its expectations against the evidence, and then adjusts each parameter using only quantities available nearby. We write CD–BP and CD–PC for CD structures fitted in these two ways.

The fourth idea is practical. A *cap* is a module attached on top of an existing, largely frozen transformer. It reads the base model’s internal states and its prediction, consults additional memories or rules, and outputs a corrected prediction. Any of the mechanisms above can be packaged as a cap, which narrows the first engineering problem a great deal, because the cap does not have to learn language from scratch.

These four names describe different aspects of a system and can be combined freely, and keeping them apart is necessary for reading the experiments correctly. GT is a way of organizing the flow of information and says nothing about the optimizer. A conventional transformer is a GT, a CD architecture is another, and a CD architecture can contain transformers inside it. CD does not mean the absence of neural weights. PC is a learning and inference procedure and is distinct from structural search, and a cap trained with BP can still carry out iterative inference at prediction time. Several experimental comparisons below differ in exactly one of these respects.

1.4 What has been tested

All experiments so far are small. They run on a CPU-scale setup that we call the sandbox, using a transformer of about 120,000 parameters that predicts text one byte at a time, mostly on the Tiny Shakespeare corpus. Their purpose is to find out which mechanisms work and which comparisons are informative before anything expensive is attempted.

The findings, in brief, are these. Finite program-discovery tasks show that search guided by past experience finds recurring rules with far fewer trials than unguided search, and that some rules cannot be found at all unless the system is allowed to test relationships among three or more variables at once. A series of text experiments shows that the criterion for accepting a new module has a large effect, and that a strict requirement for a module to pay back its storage cost quickly rejects many useful corrections. The strongest cap so far, called H2 below, lowers the prediction loss of its base transformer from about 2.90 to about 2.34 bits per byte. Most of that gain comes from letting the cap read full probability distributions from a large dictionary of character contexts. Choosing which readers to keep by a compression-first search, and adjusting trust among readers at prediction time using recent evidence, each add a smaller gain that is consistent across random seeds. PC and BP give almost identical results as learning rules. When H2 is adapted to a new domain, it loses between 12 and 14 percent of its advantage on the old one, and mixing in ten percent of old training examples during adaptation cuts that loss by about forty percent.

No experiment yet addresses decentralized exchange, integration with a symbolic reasoner, or scale. Sections 8–9 give the evidence in detail, with the limits of each comparison stated alongside it.

1.5 Relation to existing work

The ingredients have substantial precedents. Judging a compressed representation by what it preserves for a task connects to the information bottleneck [7]. Learning a library of reusable programs together with a mechanism that proposes candidates connects to DreamCoder [15]. Memory readers that supply predictive distributions connect to neural caches and nearest-neighbor language models [13, 12]. The feature-and-residual view of compression comes from work on incremental compression [16], and the organization of bounded discovery, module reuse and online adaptation from resource-bounded incremental induction (RBII) [17]. We do not claim that the individual ingredients are new. What is offered here is a synthesis with explicit interfaces and guarantees, and a small experimental investigation of it.

2 Background in brief

This section collects the standard ideas used later. It is meant for readers who do not work with language models day to day, and others can skip it without loss.

Next-token prediction. A language model reads a sequence of *tokens* and outputs a probability for every possible next token. Tokens can be words, pieces of words, or single characters. In all the text experiments below a token is one byte, so there are 256 possible values and the model outputs 256 probabilities at each step. The text the model has already seen is called the *prefix* or context. A model is *causal*, or autoregressive, if its prediction for a position depends only on earlier positions. Much care in the experiments goes into making sure that no component ever looks, even indirectly, at the token it is supposed to predict.

Measuring prediction in bits. If the model assigned probability p to the token that did occur, its loss on that token is $-\log_2 p$ bits: one bit for probability one half, two bits for one quarter, and so on. Averaged over a text, this gives the *cross entropy*. When tokens are bytes the unit is *bits per byte* (bpb), and lower is better. A model that knows nothing and spreads its probability evenly over 256 bytes scores 8 bpb. The small transformer used here scores about 2.9 bpb on Shakespeare, and good count-based models of character sequences reach somewhat lower values. For scale, a change of 0.5 bpb is a large improvement in this setting and a change of 0.005 bpb is small, though it can still be measured reliably when it appears in every repetition. The measure is closely related to data compression, since a predictor with a loss of L bits can in principle be turned into a code that stores the text in about L bits. Section 4 relies on that connection.

The transformer layer. A transformer keeps one vector of numbers for each position in the text and refines these vectors layer by layer [1]. Each layer has two steps. In the *attention* step, each position computes a *query* vector describing what it is looking for, and every earlier position offers a *key* vector describing what it contains and a *value* vector holding its content. The match between a query and each key is turned by the *softmax* function into a set of positive weights that sum to one, and the position receives the correspondingly weighted average of the values. Several such *heads* operate in parallel. In the *feed-forward* step, a small network is applied separately at each position to the result. The output of each step is added to its input (a *residual* connection) and rescaled (*normalization*).

Training. The weights are fitted by *backpropagation*: the loss is computed, its derivative with respect to every weight is obtained by applying the chain rule backward through the whole computation, and each weight is moved a little in the direction that lowers the loss. The experiments use AdamW, a standard variant of this gradient descent procedure. BP is global, in the sense that the update to a weight depends on a signal passed back through everything that lies between that weight and the output.

Count-based predictors. A much older way to predict the next character is to count. An *n-gram* or *suffix* model records, for each short context seen in training text, how often each character followed it, and predicts in proportion to those counts. When a context was never seen, the model *backs off* to a shorter context. Such models are crude in some ways and very strong in others, because they remember specific strings exactly. They appear below as the *readers* that a cap consults.

Energy-based inference and predictive coding. In an energy-based network, the internal activities are found by a search. One defines an *energy*, a number that is small when every unit’s activity agrees with what the units feeding into it predict and with the available evidence, and the activities are adjusted step by step to lower it. This process is called *settling*. Predictive coding is a family of such models in which the energy is a sum of squared prediction errors. Learning then uses the settled state: each parameter is changed according to the prediction error at its own unit, with no backward pass through the whole network. Under suitable conditions these local updates approximate the gradient that BP would compute [8, 9].

Minimum description length. The *minimum description length* (MDL) principle says that the best explanation of some data is the one that allows the data to be written down most briefly, where the length of the explanation itself is counted in the total. A rule that covers many cases is worth its cost. A rule that covers two cases is usually not, because it is cheaper to list the two cases.

3 The generalized transformer

3.1 The idea: six jobs in one layer

Picture each position in a text as a small workspace holding some notes. One layer of processing does six kinds of work on every workspace.

1. It *describes the current need*: what question should this workspace answer next?
2. It *chooses sources*: which earlier positions, memories or modules might hold useful evidence?
3. It *brings back evidence* from those sources, ideally keeping track of where each piece came from.
4. It *exposes relationships*, computing features that make a useful relation easy to inspect.
5. It *reconciles* tentative interpretations when different sources disagree, if that is needed.
6. It *prepares the next state*, keeping an affordable representation of what later computations will need.

An ordinary transformer does most of this with vectors, attention weights, weighted sums and small numerical networks. The systems studied in this report are also allowed to use explicit records and small programs. In both cases the useful question about a workspace is less “what is stored in it?” than “which differences between situations can the next computation still detect from it?” The rest of this section makes that question precise.

3.2 A running example

Suppose the text so far is

The captain is Mira. The cook is Leo. The captain is _____

and the task is to predict the missing word. We treat whole words as tokens to keep the example readable, although the experiments use bytes. The six jobs look like this.

1. *Need*. The context calls for a name associated with “captain.”
2. *Routing*. A reader that matches phrases finds the earlier, completed phrase “The captain is.” A neural attention head could learn a similar notion of relevance.
3. *Transport*. The reader returns “Mira,” together with the matched phrase or a pointer to its location. The earlier name has already been observed, so reading it is legitimate. The missing word itself is out of reach.
4. *Lift*. The layer exposes a feature such as “there is an exact earlier match for this phrase,” together with the candidate word to copy, and it keeps the pairing of role with name.
5. *Reconcile*. If another reader favors “Leo,” the layer uses whatever evidence is available to decide how far to trust each. No step of this inference may inspect the missing answer.
6. *Re-encode*. The layer keeps the candidate distribution and whatever source information and bindings the next computation will need. If later queries might include “who is the cook?”, the present question about the captain does not justify deleting the cook’s binding.

The output is a probabilistic prediction. Nothing here amounts to a logical guarantee about what an author will write next. We return to this example several times, and in Section 3.7 we use it to show why the order of these operations can decide whether information survives.

3.3 Observers and the geometry of distinctions

The informal question above, which differences can still be detected, needs a definition of “difference.” We get one from the idea of an observer.

Let S be a set of possible states. An *observer* is a function $f : S \rightarrow E$ that asks a question of a state and returns an answer. The identity of the last byte, a vector of hidden activations, the truth value of a logical predicate, and a probability distribution over the next token are all possible answers. An observer *distinguishes* two states when it answers differently on them. Writing D_f for the set of pairs that f distinguishes, and $f \succeq g$ when the answer to g can be computed from the answer to f , we have

$$D_f = \{(s, s') \in S^2 : f(s) \neq f(s')\}, \quad f \succeq g \iff g = h \circ f \text{ for some } h. \quad (1)$$

So $f \succeq g$ says that f is at least as informative as g . Asking two questions together retains both answers, and the pair distinguishes exactly those states that at least one of the questions distinguishes.

When observers return vectors, they define distances as well as distinctions. For observers f_a with positive weights ω_a , let

$$d_F(s, s')^2 = \sum_a \omega_a \|f_a(s) - f_a(s')\|^2. \quad (2)$$

This measures how different two states look through the chosen questions. If the questions inspect only punctuation, two very different sentences can be at distance zero. Adding a question about the subject’s name pulls them apart. In mathematical terms d_F is a *pseudometric*, a distance that is allowed to be zero between states that are internally different, and treating zero-distance states as one state gives an

ordinary metric space. Adding further questions without removing old ones can only increase distances. All of this follows from the observation that d_F is the usual Euclidean distance after each state s is replaced by the list of its weighted answers, $s \mapsto (\sqrt{\omega_a} f_a(s))_a$.

This is the sense in which the framework is geometric: a geometry, here, is a rule for saying which situations count as similar and which as different, and the ruler is a collection of questions. At least three such geometries are in play in any layer, namely the geometry of observed features, the geometry used for routing, and the geometry of downstream behavior, and they need not agree. The query–key scores used in attention, for instance, can be asymmetric and need not be distances at all. We use the word geometric because the objects involved include metrics, couplings, measures, embeddings and quotients. We do not assume that every component is a smooth manifold.

3.4 Routing: an input-dependent way of sharing out attention

Routing decides where each position looks. Let $X = (x_1, \dots, x_n)$ hold the current states of all positions. At position i , a head h may read from a set of permitted sources $J_i \subseteq \{1, \dots, i\}$. The token still to be predicted is never in this set. Each head builds addresses and payloads from the states,

$$q_i^h = Q_h(x_i), \quad k_j^h = K_h(x_j), \quad v_j^h = V_h(x_j),$$

which play the roles of the query, key and value of Section 2. A routing rule then supplies weights

$$a_{ij}^h \geq 0, \quad \sum_{j \in J_i} a_{ij}^h = 1, \quad a_{ij}^h = 0 \quad (j \notin J_i). \quad (3)$$

Each position thus has a fixed amount of attention to distribute over its permitted sources, and row i of the weights is a probability distribution over them. We call such a table a *coupling*. The columns need not sum to one, since a source can be read by many positions or by none, so this is not the balanced kind of coupling that appears in optimal transport.

The familiar softmax rule comes out of this picture as the solution of a simple trade-off. Give each source a cost, so that cheap or relevant sources should receive more weight, and penalize distributions that commit too strongly compared with some prior expectation.

Proposition 3.1 (Softmax as regularized source selection). *For a positive prior π_i on finite J_i , finite costs c_{ij} and $\tau > 0$, the unique minimizer of*

$$\sum_j a_j c_{ij} + \tau \text{KL}(a \| \pi_i), \quad a \in \Delta(J_i),$$

$$\text{is } a_j^* = \pi_{ij} e^{-c_{ij}/\tau} / \sum_r \pi_{ir} e^{-c_{ir}/\tau}.$$

Proof. The objective equals $\tau \text{KL}(a \| a^*) - \tau \log \sum_r \pi_{ir} e^{-c_{ir}/\tau}$. Nonnegativity of KL gives the result. Here KL and the exponentials use natural logarithms. $\square$

Here $\Delta(J_i)$ is the set of probability distributions on J_i , and $\text{KL}(a \| \pi)$, the Kullback–Leibler divergence, measures how far a departs from π . The parameter τ sets how strongly early commitment to a single source is discouraged. With a uniform prior, costs $c_{ij} = -(q_i^h)^\top k_j^h / \sqrt{d_k}$ and $\tau = 1$, the minimizer is ordinary scaled dot-product attention. Other routing rules are equally admissible. A program that looks up a dictionary entry puts all the weight on the matching record, a sparse reader puts it on a few records, and a retrieval system returns a list. A learned search process need not compute a softmax at all.

3.5 Transport, aggregation, lifting and re-encoding

Once weights are chosen, evidence has to be brought back, and a design decision arises that standard attention makes silently. Before anything is averaged, the incoming evidence for head h at position i can be written as a weighted collection of items, each tagged with its source:

$$\nu_i^h = \sum_{j \in J_i} a_{ij}^h \delta_{(j, V_h(x_j))}. \quad (4)$$

The symbol $\delta_{(j,v)}$ denotes a unit of weight placed on the item “payload v , from source j .” Mathematically ν_i^h is a *tagged measure*. It keeps the selected items, their weights and their origins. It cannot restore information that the payload map V_h has already thrown away. Standard attention immediately replaces this collection by its weighted average, or *barycenter*,

$$B(\nu_i^h) = \int v d\nu_i^h(j, v) = \sum_j a_{ij}^h V_h(x_j). \quad (5)$$

Averaging is one choice of aggregation among several. Other implementations keep the records separate, or pass along references to executable objects. Treating attention in terms of measures has established precedents [6].

Next, the local state and the incoming evidence are placed side by side, and derived features are added:

$$z_i = (x_i, \nu_i^1, \dots, \nu_i^H), \quad \Phi_G(z_i) = (z_i, g_1(z_i), \dots, g_m(z_i)). \quad (6)$$

We call Φ_G a *lift*. Because its first coordinate is z_i itself, nothing is lost, and in mathematical terms the map is injective. The derived features $g_1, \dots, g_m$ add no information beyond what the complete state already contains. What they change is cost: a relation that was expensive to work out becomes cheap to use. A database index is a good comparison, since it adds no facts to the database and still changes what can be done with it quickly.

Finally, let $\mathcal{S}$ be an optional reconciliation step, which runs a finite inference procedure and by default does nothing, and let R re-encode the result as the next state. A GT layer is the composition

$$x_i^+ = R \left(\mathcal{S} \left(\Phi_G \left[x_i, \left\{ \sum_j a_{ij}^h \delta_{(j, V_h(x_j))} \right\}_{h=1}^H \right] \right) \right). \quad (7)$$

Read from the inside out, the layer routes and transports evidence, places it beside the local state, lifts, reconciles, and re-encodes (Figure 1). Persistent state belonging to a module can be supplied as one more permitted input. Layers can be stacked, so that later layers refine both the routing and the representation. The factorization describes the logic of the computation, and an efficient implementation is free to avoid building every intermediate measure or keeping every coordinate.

It is fair to ask whether this description says anything, since almost any function could be described as “route, then transform.” It does impose restrictions. All access from one position to another has to be declared through the routing weights and payloads. The local maps consume that evidence and nothing else. The order of observation is explicit, so that a violation of causality is visible as such. And the costs of reading, storing, inferring and rewriting are counted separately. These restrictions are what make the framework a guide to design and measurement.

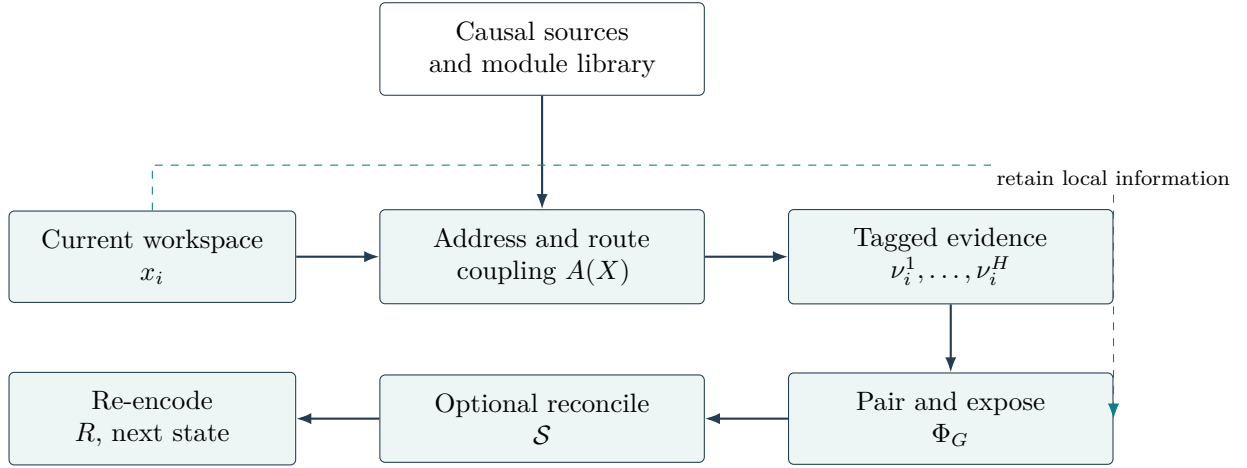


Figure 1: The GT layer. Where evidence is aggregated, and where information is given up, are explicit design choices inside the local maps. The dashed arrow means that local information is logically available to the left, which does not require copying a full state in memory.

3.6 The ordinary transformer is an exact special case

Ignoring dropout, a post-normalization transformer block computes

$$\begin{aligned}
 m_i^h &= \sum_j \text{softmax}_j \left((W_Q^h x_i)^\top W_K^h x_j / \sqrt{d_k} \right) W_V^h x_j, \\
 u_i &= N_1 \left(x_i + W_O [m_i^1; \dots; m_i^H] \right), \\
 x_i^+ &= N_2(u_i + W_2 \sigma(W_1 u_i + b_1) + b_2),
 \end{aligned} \tag{8}$$

with causal masking understood. The first line is attention, the second adds the attention output to the state and normalizes, and the third is the feed-forward step [1].

Proposition 3.2 (Transformer containment). *Equation (8) is a GT layer: use the stated query/key/value maps, take barycenters of the tagged measures, expose the intermediate local computations, set S to identity, and choose R to return the final normalized residual expression.*

Proof. Substituting these maps into Equation (7) gives each line of Equation (8) in order. Pre-normalization and other deterministic block variants alter the local maps while leaving this containment argument unchanged. $\square$

Two cautions apply. The containment does not show that standard normalization keeps everything later computations need. And a residual sum does not by itself preserve its two summands separately, because the map $(x, m) \mapsto x + m$ sends many different pairs to the same result. A trained network, restricted to the states it can reach, may behave better than this worst case suggests. A fixed vector width, taken alone, proves neither that information is lost nor that it is kept.

3.7 Why the order of operations can decide what survives

The running example shows concretely how averaging too early can destroy a relationship. Compare two memories. Memory A holds the records captain–Mira and cook–Leo. Memory B holds captain–Leo and cook–Mira. Suppose each record is encoded by concatenating a one-hot code for the role with a one-hot

GT operation	Conventional transformer	Ideal CD realization
Observe	Hidden activations and projections	Typed numerical or symbolic observers
Route	Query-key attention	Causal lookup, search, sparse coupling
Transport	Values, usually immediately averaged	Tagged evidence or references; aggregation optional
Expose	Nonlinear feature computations	Reusable executable distinctions and dependencies
Reconcile	Optional iterative architecture	Optional PC, constraint, or other inference
Re-encode	Projection, residual sum, normalization	Cost-aware representation sufficient for declared uses
Learn	Usually fit fixed-architecture parameters	Fit parameters and search over executable structure

Table 2: Two implementations of the same interface. Preserving what later uses require is an additional demand made by the ideal CD design. It does not follow from calling something a GT.

code for the name, and that the layer takes a uniform average of the two records. The average is the same for A and for B. It still says which roles and which names occur, and it has lost who holds which role. No later function applied to that average, however nonlinear, can recover the missing binding, because the two memories have become the same input.

The binding survives if the layer routes selectively, reading only the captain’s record, or if it computes a joint role-and-name feature before averaging. Several attention heads can achieve the same effect. The example illustrates one way an interface can lose information, and it should not be read as a claim that transformers generally lose such bindings. Table 2 sets out how a conventional transformer and an ideal CD system carry out each GT operation.

4 Compression-driven distinction networks

4.1 The idea: rules, exceptions, and a better index

A CD workspace is a small executable explanation of what it has observed. It has three parts: reusable features or rules, the exceptions those rules fail to explain, and a way of finding the right pieces quickly. When errors keep recurring, the system tries out a new explanation. The candidate might be a lookup on the last few characters, a copy rule, a conjunction of two existing features, a relation between them, or a small composition of known operations. The system refits the numerical parts that the change affects, checks whether prediction has improved, and keeps the discoveries that help.

The running example shows the range of options. Suppose the system repeatedly sees sentences that assign names to roles. Storing every sentence preserves all the evidence. Adding an index from each role to its assigned name makes a later question cheaper to answer. Learning a reusable program, “find the most recent assignment for this role,” can in some contexts make the index unnecessary. Each step shortens the description, and a shorter description is an improvement only if it stays accurate and remains affordable to execute. A detail that one question lets us discard may still be needed by another.

Three earlier lines of work meet in this design. The calculus of distinctions of Section 3 says what a representation must keep. Incremental compression [16] supplies the view of learning as the repeated extraction of a feature together with a residual that the feature leaves unexplained. Resource-bounded incremental induction [17] supplies the organization of bounded search, module reuse and online adaptation. The definitions needed from each are given below, and no familiarity with those sources is

assumed.

4.2 What a compressed state must preserve

Compression discards detail, and the first thing to settle is which detail may be discarded safely. The answer used here is relative to later use: a state may forget whatever no later computation will miss.

To state this, fix a family $\mathcal{C}$ of later computations, which we call *continuations*. A continuation might answer a question, make a prediction, invoke a module, or carry out the next round of routing. Let $p_c(\cdot \mid z)$ be the probability distribution over outputs that continuation c produces from state z . The *behavioral distance* between two states is the largest difference any continuation can show between them,

$$d_{\mathcal{C}}(z, z') = \sup_{c \in \mathcal{C}} \text{TV}(p_c(\cdot \mid z), p_c(\cdot \mid z')), \quad \text{TV}(p, q) = \frac{1}{2} \sum_y |p(y) - q(y)|, \quad (9)$$

where TV, the total variation distance, is the largest difference in probability that two distributions assign to any event. Like d_F earlier, $d_{\mathcal{C}}$ is a pseudometric. Merging all states at behavioral distance zero gives the *behavioral quotient* $Q_{\mathcal{C}}$, the coarsest summary that every continuation in the family can still use without loss. The family has to be declared independently of any proposed compression. Otherwise one could call any compression lossless by declaring only the uses that happen to survive it.

The next two results make the connection between compression and behavioral distance exact. A re-encoding r sends each state to a code, and the set of states sharing a code u is called the *fiber* $r^{-1}(u)$. The *diameter* of a fiber is the largest behavioral distance between two of its members.

Theorem 4.1 (Sufficiency as zero-diameter fibers). *For finite Z , a re-encoding $r : Z \rightarrow U$ preserves every continuation in $\mathcal{C}$ exactly if and only if every fiber $r^{-1}(u)$ has $d_{\mathcal{C}}$ -diameter zero. Equivalently, each p_c factors through r . The canonical quotient $Q_{\mathcal{C}}$ is recoverable from every sufficient r .*

Proof. If $p_c = \hat{p}_c \circ r$, states with equal codes have equal continuation outputs. Conversely, if outputs are constant on a fiber, define $\hat{p}_c$ using any representative. The same argument maps each sufficient code to its unique behavioral class. $\square$

Proposition 4.2 (Approximate fibers and unavoidable loss). *If a decoder approximates every $p_c(\cdot \mid z)$ within TV error ϵ , each fiber has diameter at most 2ϵ . Conversely, fibers of diameter at most ϵ admit representative-based decoders with error at most ϵ . In particular, identifying two states at behavioral distance δ forces uniform error at least $\delta/2$ for any decoder using only that code.*

Proof. For the first claim, insert the common decoded distribution between two true distributions and use the triangle inequality. For the converse, choose a representative of each nonempty fiber. The last claim is the contrapositive of the first. $\square$

In words: two situations can share one compact note exactly when every later question we care about can still be answered from that note. If situations that behave differently are given the same note, some later question must be answered wrongly for at least one of them, and the size of the unavoidable error is half their behavioral distance. The ideal quotient draws the coarsest set of distinctions compatible with the declared questions. Two limits should be kept in mind. The quotient does not tell us the shortest program for writing the note down, and it offers no protection to questions that were never included in $\mathcal{C}$.

The ideal CD layer is a GT layer whose final step is this quotient:

$$x_i^+ = Q_{\mathcal{C}_i}(\mathcal{S}(\text{Expose}_{G_i}[x_i \sqcup (A_i(X) \triangleright V(X))])). \quad (10)$$

Here $\sqcup$ keeps the local and the incoming evidence side by side, and $\triangleright$ applies the read rule given by the routing weights. When PC is not used, $\mathcal{S}$ can be the identity. What the CD layer adds to the general GT layer is the requirement that the final re-encoding preserve the chosen continuations. A real implementation can only approximate this, using a finite family of observers and a finite budget.

There is also a local, differentiable version of behavioral sensitivity, useful when the state is a vector and the predictive distributions are smooth and positive. With fixed weights ρ_c on the continuations, define

$$G(z) = \sum_c \rho_c \sum_y \frac{\nabla p_c(y | z) \nabla p_c(y | z)^\top}{p_c(y | z)}. \quad (11)$$

This is a Fisher information matrix. For twice continuously differentiable probabilities on a finite alphabet, it gives the cost of a small displacement Δ of the state:

$$\sum_c \rho_c \text{KL}(p_c(\cdot | z) \| p_c(\cdot | z + \Delta)) = \frac{1}{2} \Delta^\top G(z) \Delta + o(\|\Delta\|^2).$$

Appendix A gives the derivation. Directions in which this quadratic cost is large are directions in which a small change to the state strongly changes predictions. A direction of zero cost indicates insensitivity to first order only and does not certify that two distant states are equivalent. This local geometry suggests weighting approximation errors by G , an idea the experiments have not yet tried.

4.3 Executable feature-and-residual representations

We now turn from what should be preserved to how a CD system represents data. Let data D be represented by a program graph M with fitted parameters θ , a set of features F , a residual R_D and an index I . In the lossless version of the design, the data can be reconstructed exactly:

$$D = \text{Decode}(M, \theta, F, R_D, I). \quad (12)$$

The residual records whatever the reusable structure fails to predict, and corresponds to the “exceptions” of Section 4.1. In the lossy, task-relative version, Equation (12) is replaced by requirements on the outputs of the declared continuations. The two versions are different contracts. A predictor can be fast and accurate without being able to reconstruct every original observation.

An idealized objective charges for everything:

$$J = L(M, \theta, I | \mathcal{L}) + L(D | M, \theta, I) + \lambda C_{\text{run}} + \gamma C_{\text{search}} + \mu \mathcal{D}_C. \quad (13)$$

The first term is the length of the model, given a fixed and versioned shared library $\mathcal{L}$ of modules. The second is the length of the data given the model. These two terms are the MDL criterion of Section 2. The remaining terms charge for running cost, for search cost, and for the behavioral distortion $\mathcal{D}_C$ suffered by the declared continuations. The coefficients convert each cost into common units, and running time is not literally a number of bits unless a convention says so. A full accounting includes stored residuals, dictionaries, replay data and the state of the search guide. A reference to a shared module is cheap only if the shared storage and the cost of accessing it are charged somewhere.

For a causal predictor with properly normalized probabilities, the ideal code length of the data is

$$L_p(D) = - \sum_t \log_2 p(y_t | x_{<t}).$$

An actual compressed file needs more: framing, transmission of the model if the receiver does not already have it, probabilities of finite precision, and coding overhead. The bits-per-byte figures reported below are predictive cross entropies, and should not be read as sizes of compressed files.

Searching over all programs for the best representation cannot be carried out in practice. A working CD system chooses a finite grammar of structures, finite budgets for fitting parameters, and a bounded set of candidate modifications. Refitting after each structural change is essential, because the value of a feature depends on how it interacts with the other features present, and its correlation with the target in isolation can be a poor guide.

4.4 When is a new module worth keeping?

The objective in Equation (13) suggests a simple admission rule: accept a module only if the bits it saves exceed the bits needed to store it. The earliest prototypes applied this rule over a short assumed deployment horizon, and it turned out to be too strict, rejecting corrections that improved prediction consistently. A calculation shows why. If a module occupies B bytes and saves $g > 0$ bpb on future data, the amount of future data needed for the saving to repay the storage is

$$T_{\text{break}} = \frac{8B}{g}. \quad (14)$$

A module of 10,000 bytes that saves 0.01 bpb needs about eight million predicted bytes to repay its storage, before any search or execution cost is counted and assuming the gain persists. With the short deployment horizons assumed in the early prototypes, very few modules could pass such a test.

Later prototypes therefore select candidates by predictive improvement under explicit ceilings on memory and work, and report the repayment of code cost as a separate accounting measure. They also ask a slightly different question about each candidate. The question is whether adding the candidate to a mixture improves the mixture, which can be true of a candidate that is a weaker predictor than the incumbent when taken alone:

$$U(a) = L_A(P_{\text{control}}) - \min_{\alpha \in [0,1]} L_A((1 - \alpha)P_{\text{control}} + \alpha P_a). \quad (15)$$

Here P_a is the candidate, P_{control} is a control predictor given the same opportunity to refit but no structural change, and L_A is the loss on a development set A . The blend weight and the candidate are chosen on A . The outcome that is recorded, with its sign, for training the search guide is measured on a separate set B . Since $\alpha = 0$ is allowed, the optimized gain on A cannot be negative, but the measured gain on B can be, and it is kept as measured.

This is a controlled relaxation of a short-horizon MDL gate and should not be mistaken for a finding that model cost is irrelevant. It admits useful candidates, keeps hard ceilings on capacity, and treats long-run return on resources as a separate question. One related idea was tested and set aside. Training each candidate with a mixture-aware objective often did worse, at the short fitting budgets available, than fitting each expert in the ordinary way and assessing its contribution to the mixture afterwards. Fitting an expert and evaluating its contribution to an ensemble need not use the same objective.

4.5 Bounded discovery, reuse and adaptation

Following RBII, the design separates three activities. A discovery process proposes executable programs or fragments of a program graph. A library stores the accepted modules with their definitions, dependencies, fitted state and the record of where they came from. An execution process selects modules according to context, applies them, and may adapt them. A reader discovered once does not have to be rediscovered each time it is needed.

It is convenient to write the state of the whole machine at time t as

$$\Xi_t = (\mathcal{L}_t, M_t, \theta_t, H_t, I_t),$$

where H_t is the record of previous searches. For each observation o_t , the system first predicts using Ξ_t and the prefix available so far, and only afterwards uses the revealed outcome to update its mutable state. The order is essential both for coding and for honest evaluation. A system that adapts on the target and then reports the adapted prediction as if it had been made beforehand has leaked the answer into its own score.

Updates made to a module in different contexts need not commute, which means that applying two updates in the opposite order can leave the module in a different state. Sharing a module between contexts safely therefore requires one of three things: isolating versions, checking that protected behavior is preserved, or bounding the behavioral difference between the two orders. A bound of the last kind is useful only if it is measured in a relevant behavioral metric and if its amplification by later computation is controlled. Appendix A.2 makes this precise. The experiments so far implement only bounded pieces of this program.

4.6 Dependency towers and a search guide that learns from history

Structural search is expensive, and the design tries to make it cheaper in two ways. It organizes what the system measures about its data, and it learns from the outcomes of previous searches.

The first device is the *dependency tower*. Let K describe the state of the data or of the search. Choose nested families $\mathcal{O}_{k,b}$ of executable observations, indexed by the order k of the dependencies they examine and by a resource budget b . For observations with bounded real values, let

$$d_{k,b}(K, K') = \sup_{O \in \mathcal{O}_{k,b}} |O(K) - O(K')|. \quad (16)$$

At level (k, b) , two states count as different only if some affordable observation of order at most k tells them apart. A tower might climb from counts of individual features, to interactions between pairs, to selected interactions among three or more. Order and cost are separate things. A single parity test involves many variables and is cheap to run, whereas enumerating all pairs over a large vocabulary involves only two variables at a time and is expensive.

The second device is a search guide, modeled on the support tensor logic machine (STLM), a way of scoring logical or program *templates* that still contain unfilled slots without enumerating every way of filling them. The guide scores a candidate expression e in stages:

$$e \mapsto \text{canon}(e) \mapsto \Phi_e(K) \mapsto \Psi(K, e, H) \mapsto \hat{V}_\phi(K, e). \quad (17)$$

The expression is first put in a canonical form, so that equivalent ways of writing the same template are recognized as one, whenever equivalence can be decided within the chosen grammar. It is then evaluated against the data by Φ_e , which may be a tensor of predicate values, a selected contraction of such tensors, or another structured evaluation. Next, Ψ gathers descriptors of the template’s structure, the tower observations, support counts, statistics of the current residual, the search history and the cost. Finally a learned regressor $\hat{V}_\phi$, which may be a support-vector machine, a learner over explicit features, or a neural approximation, estimates how useful the candidate will be.

An action proposed by the guide can specify far more than the name of a feature:

$$a = (e, \text{bindings}, \theta_0, \text{refit neighborhood}, \text{observation schedule}, \text{budget}).$$

History can therefore shape the starting conditions of a search: which motif to try, where to attach it, how to initialize its parameters, which neighboring modules to refit along with it, and which missing statistic to measure first. Successes, failures, results of exactly zero, and attempts that ran out of budget

all carry information. A timeout tells us about one attempt made with one budget and does not show that the template is useless.

A useful picture is a mechanic’s notebook. It records which repairs worked under which circumstances, the unsuccessful ones included. A new fault calls to mind several past cases, so the mechanic begins with sensible repairs and sensible settings. The repair still has to be carried out and tested. The notebook’s prediction that a repair will succeed does not certify that the machine now works.

That last distinction is built into the design. The record of *verified* observations defines the set of states of the world still compatible with everything established so far,

$$\mathcal{W}_t = \{K : K \text{ satisfies all sound constraints recorded so far}\}.$$

Each new sound observation shrinks this set. A predicted outcome can be used to schedule a search but must never be entered as a proven constraint, and statistical intervals carry their own coverage assumptions and their own accounting of failures. This separation is what connects the guide to the exact bounds of Appendix B.

A guide that followed only its own predictions would never discover where they are wrong, so proposals are drawn from a mixture of the learned distribution $q_{\phi,t}$ and a broad default distribution $q_{0,t}$:

$$q_t = (1 - \varepsilon_t)q_{\phi,t} + \varepsilon_t q_{0,t}. \quad (18)$$

If a fixed discoverable action has probability at least $p > 0$ of being proposed on each trial, the probability that it is missed in n trials is at most $(1 - p)^n$. This elementary bound does not ensure discovery in an infinite and changing space of programs, and it says nothing about whether refitting will succeed or the gain will be positive.

4.7 Why higher-order observations are sometimes indispensable

The tower allows for high-order observations because some regularities cannot be seen at any lower order. Let A and B be independent fair bits and let $C = A \oplus B$ be their exclusive-or. Any two of the three variables are statistically independent, even though the three together satisfy a deterministic rule. Statistics of pairs cannot reveal that rule. The same holds in general: the uniform distributions on the even-parity and the odd-parity strings in $\{0, 1\}^d$ have identical marginals on every proper subset of positions, because each assignment to $k < d$ positions has 2^{d-k-1} completions of either parity.

This motivates a tower that is sparse and selectively deep. The architecture should be able to propose a compact high-order program when lower-order descriptions fail, without ever building a full joint table over all variables. Appendix C works through a finite noisy-parity example in which one additional high-order observation turns an inconclusive compression proposal into a certified improvement.

4.8 Training the guide: warm-up, label quality, and richer features

The guide learns from the outcomes of searches, and the quantity and quality of those outcomes limit what it can learn. A thousand candidate trials do not necessarily amount to a thousand independent search situations. Candidates evaluated against the same incumbent share its residuals, its library and its text. The capacity of the guide should therefore be governed by how many distinct states or search trajectories support it, and its calibration should be checked on held-out groups of trials, never on individual trials drawn from the same group. Sparse higher-order features of the guide can be shrunk toward a simpler guide until the data support them.

The precision of each outcome label also limits the guide. As a rough illustration, if the paired difference in loss between a candidate and its control had standard deviation σ per target, and targets were

Algorithm 1 A bounded CD discovery episode

Require: Current predictor/library, fitting data, selection A , outcome data B , work budget

- 1: Freeze versions and evaluation roles; retain the incumbent as an option
- 2: Propose templates, bindings and initial parameters from history plus exploration
- 3: **while** budget remains **do**
- 4: Select an observation or candidate using estimated utility and remaining cost
- 5: **if** additional evidence is requested **then**
- 6: Acquire the observation; record its scope, uncertainty, provenance and cost
- 7: **else**
- 8: Clone an incumbent; execute a bounded candidate refit
- 9: Fit a matched no-structure-change control
- 10: Evaluate deployed mixture utility on A , charging storage and work limits
- 11: Retain the candidate if admissible and useful; otherwise retain the control/incumbent
- 12: Record signed outcome on B for later guide learning
- 13: **end if**
- 14: **end while**
- 15: Keep a complete trace; consolidate in a separate protected step if worthwhile

independent, then estimating the mean difference to within an approximate 95% half-width e would need about $n \approx (1.96\sigma/e)^2$ targets. With $\sigma = 0.05$ bpb, a half-width of $e = 0.001$ needs about 9,600 targets, and $e = 0.0002$ needs about 240,100. Correlated text, adaptive selection and multiple candidates all change the calculation. These figures indicate the scale involved and are not a power analysis of the experiments.

The practical proposal for warming up a guide is to mix varied, independently constructed states with real search trajectories, to probe deliberately some candidates that look uncertain or unpromising, to use matched controls and signed outcomes, and to enlarge the guide only when its ranking of held-out candidates improves. Extra search early on can be an investment that is reused. If warm-up costs W units of work and afterwards saves $s > 0$ units at each comparable search stage, it breaks even in compute after W/s stages. The experiments do not yet establish a generally positive value of s .

What has been built so far. The earlier text caps described in Section 8 implement finite graphs of observers and a learned ranking of candidates by history, in the spirit of this section. They use mainly tree-based regression over structured descriptors and are not full support tensor logic machines. The most recent cap, H2, does not include the learned guide. It tests proposals ranked by the current residual, and the reuse of a structure found during development.

Algorithm 1 summarizes one episode of bounded discovery.

5 Caps: attaching CD to an existing transformer

5.1 The idea

A cap is a specialist workspace attached to a model that is already capable. The base model reads the text and supplies a default prediction. The cap consults additional memories or rules and decides whether they justify a correction. The new mechanism can then concentrate on the base model’s remaining errors and on new knowledge, and does not have to learn language from the beginning.

In the running example, the base might supply a plausible distribution over names. A copy reader notices that the phrase “The captain is” was completed earlier in the text, and the cap uses that evidence to

raise the probability of Mira. When the memory has nothing relevant to offer, a well-trained cap should leave the base prediction almost unchanged. Whether it makes that decision correctly has to be tested, since neither a small parameter count nor a frozen base guarantees it.

5.2 Two ways for a cap to write its output

Let $p_B(y | x) > 0$ be the base model’s distribution over the next token y given the prefix x , and let $h_B(x)$ denote the hidden states of the base that the cap is allowed to read. There are two natural interfaces through which a cap can influence the output, and both yield properly normalized probabilities.

In the first, the cap produces a correction score $s_C(x, y)$, measured in bits, for each possible next token, and the scores tilt the base distribution:

$$p_C(y | x) = \frac{p_B(y | x) 2^{s_C(x, y)}}{Z_C(x)}, \quad Z_C(x) = \sum_y p_B(y | x) 2^{s_C(x, y)}. \quad (19)$$

A correction of zero returns the base distribution exactly. On a particular target, the number of bits the cap saves is

$$g_C(x, y) = \log_2 \frac{p_C(y | x)}{p_B(y | x)} = s_C(x, y) - \log_2 Z_C(x). \quad (20)$$

If the scores are bounded, $|s_C| \leq a$, then $Z_C \in [2^{-a}, 2^a]$ and therefore $|g_C| \leq 2a$. The gain or loss on any single target is thus bounded without any assumption that the base gives every outcome a large probability. The normalizer Z_C is a sum over the whole vocabulary and takes real work to compute. A shortcut that corrects only the top few candidates and skips the normalization does not inherit this bound.

In the second interface, which the most recent cap uses, every reader emits a complete distribution of its own, and the cap outputs a weighted mixture:

$$p_C(y | x) = \sum_{j \in \mathcal{A}} w_j(x) q_j(y | x), \quad w(x) \in \Delta(\mathcal{A}), \quad q_0 = p_B. \quad (21)$$

Here $\mathcal{A}$ is the set of active readers, the base model is included as reader zero, and the weights $w_j(x)$ are nonnegative and sum to one. With this interface the cap learns which evidence to trust in which context, which is a different task from learning to produce a small vector of corrections. The two interfaces can represent overlapping families of predictors, but they differ in how easily they are fitted and in their memory cost. Figure 2 shows the overall arrangement.

Some accounting caveats apply to any cap. A claim of lossless coding that is conditional on a shared base still has to pay for the cap’s new programs, fitted parameters and memory. Freezing the base avoids differentiating through it while the cap is fitted, provided the states the cap reads are treated as fixed inputs. It does not make free the forward passes through the base, the storage of its hidden states, normalization over the vocabulary, or retrieval. And if a cap writes into early layers of the base, the cached states of later layers become invalid and have to be recomputed.

5.3 Composable modules need to be trained together

Suppose several cap modules $C_1, \dots, C_m$ read a stable workspace and each supplies a correction, or an energy term in the sense of Section 2. Modules that are each useful alone are not necessarily useful together. Two may duplicate the same evidence, contradict one another, or depend on incompatible internal coordinates. Good specialists do not automatically make a good team. They need agreed input formats, a way of reporting their confidence and their sources, and practice working in different

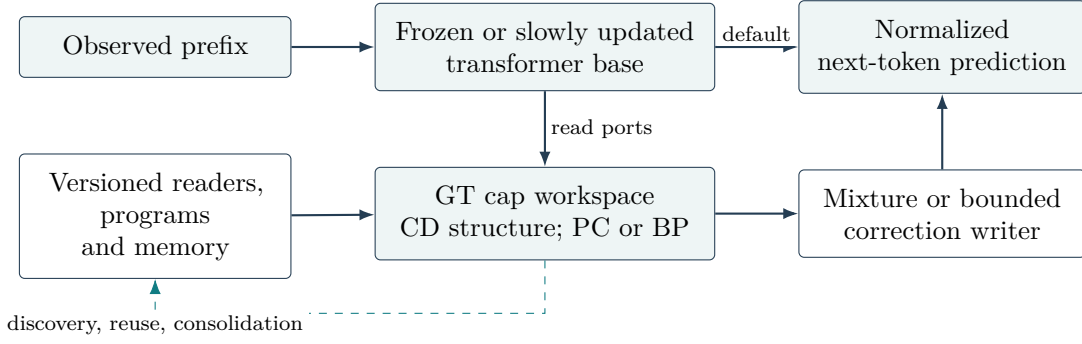


Figure 2: The cap arrangement. In the first scaling target the cap reads states of the base, with no gradient flowing back into the base, and writes either a correction or a mixture at the output. Writing into the interior of the base is a separate and more expensive extension.

combinations. A shared workspace gives them a place to meet, and only training and testing can show whether they cooperate.

One possible design for a shared state gives the active subset A of modules the combined energy

$$E_A(z; x) = E_B(z; x) + \sum_{j \in A} \alpha_j E_j(z; R_j h_B(x), M_j), \quad (22)$$

where E_B is a base energy, R_j selects the part of the base state that module j reads, and M_j is the module’s memory. This says how the modules meet. It does not show that the combined energy has a unique minimum or that the result is independent of the order in which the modules settle. A training objective aimed at composition would sample subsets of modules and, where order is relevant, execution orders π :

$$\mathbb{E}_{A, \pi, (x, y)} [-\log p_{A, \pi}(y \mid x)] + \lambda \mathcal{R}_{\text{old}} + \eta \mathcal{R}_{\text{duplicate}} + \xi \mathcal{R}_{\text{order}}. \quad (23)$$

The three penalties assess, on declared test probes, whether protected old behavior is kept, whether evidence is being counted twice, and how sensitive the result is to order. This is a proposed extension, and no benchmark of composed caps has been completed. While a new combined representation is on trial, the old branches should stay available.

5.4 Distilling a base so that caps fit it well

So far the base has been taken as given. It could instead be prepared for its caps. Distillation, in which a smaller student model is trained to imitate a larger teacher [10], can be extended so that the student is optimized both for its predictive behavior and for how accessible it makes useful evidence to a cap. Writing ψ for the parameters of the student base and ϕ for an initial cap, a proposed objective is

$$\min_{\psi, \phi} \mathbb{E}_x \text{KL}(p_T(\cdot \mid x) \parallel p_{B_\psi}(\cdot \mid x)) + \lambda \mathbb{E}_e L_{Q_e} \left(C_{\text{Adapt}_K(\phi, S_e; \psi)} \circ B_\psi \right) + \gamma \mathcal{R}_{\text{interface}} + \eta \mathcal{R}_{\text{retention}}. \quad (24)$$

The first term is conventional distillation from the teacher distribution p_T . The second says that caps should adapt well. In each episode e the cap is adapted for K bounded steps on a support set S_e and then scored on separate query data Q_e . The interface penalty keeps the read ports stable and makes selected symbolic or probabilistic observations cheap to decode from the base. The retention penalty scores old behaviors and combinations of caps.

The aim differs from simply shrinking a teacher. It is to obtain a substrate on which useful later changes are inexpensive. A teacher cannot pass on evidence it never had, so new capabilities must still come

from experience, from additional computation, or from structure that has been made newly accessible. Whenever the interface changes, memory keys, modules and caches have to be migrated explicitly. No experiment on joint distillation of a base and its cap has been carried out.

6 Learning inside the structure: predictive coding and backpropagation

6.1 The idea: revise the answer, the habit, or the machinery

A learner can respond to an error in three different ways. It can revise a *temporary interpretation*: perhaps the copy reader deserves more trust on this particular prefix. It can revise a *persistent habit*: perhaps its gate should generally trust that reader in circumstances like these. Or it can revise the *available machinery*: perhaps it needs a new reader, or a new relation between existing features.

The three components of the design correspond to these three responses. Predictive coding separates temporary activities from slowly learned parameters, and uses discrepancies between prediction and evidence to settle the activities. CD supplies the process that revises structure. The persistent parameters can be fitted by PC’s local rule, or by BP applied to the same computation. The research question is which combination gives the best trade-off among prediction, retention, discovery and compute. The existence of a local update formula is no evidence that PC learns better than BP. That has to be measured against a matched BP control, and Sections 8 and 9 report such measurements.

6.2 An energy-based realization of the GT layer

Fix a computational graph M whose nodes v carry activity vectors z_v . A useful family of energies is

$$E_0(z; x, \theta, M) = \sum_{v \in M} \frac{1}{2} \|z_v - f_v(z_{\text{pa}(v)}; \theta_v)\|_{\Lambda_v}^2 + E_{\text{evidence}}(z; x, M). \quad (25)$$

Each node v has a function f_v that predicts its activity from the activities of its parent nodes $\text{pa}(v)$, and the first term adds up the squared prediction errors. The matrices Λ_v are positive semidefinite *precisions* that say how seriously each error is taken, with $\|r\|_{\Lambda}^2 = r^\top \Lambda r$. The evidence term ties the activities to observations and uses only information available at prediction time. Running a settling procedure for K steps from an initial state z^0 gives $z^K = \text{Settle}_K(E_0, z^0)$, and a readout turns z^K into a normalized prediction. This realizes the optional reconciliation map $\mathcal{S}$ of Equation (7).

The graph may itself be built from GT layers, and CD may propose changes to its readers, edges and local feature maps. A standalone hybrid would learn its representations as well as its readout. A hybrid cap reads the representations of a base model, holds them fixed, and learns only the additional graph. The experiments investigate the second arrangement.

For a fixed graph there are two ways to learn the parameters. In CD–BP, one differentiates through the finite inference procedure that is run in practice:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\text{Readout}(z_{\theta}^K), y). \quad (26)$$

A model trained this way still settles at prediction time. A simpler BP cap could instead use a direct feed-forward readout with no settling. These are different controls, and the experiments keep them under separate names.

In the PC variant used here, which follows equilibrium propagation, the target enters through a *nudged* energy

$$E_{\beta}(z) = E_0(z) + \beta C(z, y), \quad \beta > 0, \quad (27)$$

where C is the cost of the prediction given the true target y . Prediction always uses the *free* state obtained by settling E_0 . The true target affects only the nudged state, which is used only in training. The free phase gives the system’s unaided answer. The nudged phase gently shows it the direction in which the answer should have moved. Comparing the two states reveals which local habits contributed to the error, and those differences drive the changes to the persistent parameters. The construction is related to equilibrium propagation and to results showing that predictive coding can approximate BP [8, 9]. We do not suggest that every algorithm called predictive coding is equivalent to it.

The following theorem states the sense in which the local rule recovers the true gradient.

Theorem 6.1 (Weak-nudge gradient under equilibrium assumptions). *Suppose $E_0(z, \theta)$ and $C(z)$ are sufficiently smooth, $z_0(\theta)$ is a smooth stationary branch of E_0 , and its state Hessian $H = \partial_{zz}^2 E_0(z_0, \theta)$ is positive definite. Let z_β be the corresponding stationary branch of $E_0 + \beta C$. Then*

$$\nabla_\theta C(z_0(\theta)) = \lim_{\beta \rightarrow 0} \frac{\partial_\theta E_0(z_\beta, \theta) - \partial_\theta E_0(z_0, \theta)}{\beta}. \quad (28)$$

Proof. Implicit differentiation gives $\partial_\beta z_\beta|_0 = -H^{-1} \nabla_z C$ and $\partial_\theta z_0 = -H^{-1} \partial_{z\theta}^2 E_0$. Expanding the numerator yields $-\partial_{\beta z}^2 E_0 H^{-1} \nabla_z C$, the chain-rule gradient of $C(z_0(\theta))$. $\square$

So if the system settles accurately and the nudge is small, the advice given by the local rule approaches the advice given by ordinary differentiation. The qualifications are part of the statement. If the cost depends explicitly on θ , its direct derivative has to be added. A finite nudge and incomplete settling both introduce error. BP through K settling steps differentiates those K steps, whereas the theorem concerns the exact equilibrium. When a numerical audit finds PC updates close to BP gradients, that is good evidence that the implementation is correct. It shows neither that PC learns better nor that it would run faster on any hardware.

6.3 How prediction errors can inform structural search

Residual errors that are persistent and structured suggest that an observer or an interaction is missing, and the residuals left by PC can be used to rank candidate additions cheaply. Suppose a candidate writer ϕ_a , entering with coefficient u , changes the loss locally according to the quadratic model

$$\Delta L(u) \approx g_a^\top u + \frac{1}{2} u^\top H_a u.$$

If $H_a + \lambda I$ is positive definite, the best coefficient under this model with a penalty λ predicts an improvement of

$$\hat{G}_a = \frac{1}{2} g_a^\top (H_a + \lambda I)^{-1} g_a. \quad (29)$$

This is the exact optimum of the quadratic surrogate, and carries no guarantee for the original nonlinear model. Its use is to rank proposals cheaply, after which the best are refitted properly against a matched control. Correlations among residuals, the mistakes of individual readers, and patterns of interaction can also serve as tower features for the search guide. The most recent cap uses a simpler screen, which directly measures the gain from inserting a candidate into the mixture and does not attempt the general problem of estimating structural Hessians.

6.4 Three timescales, and learning while serving

The three kinds of revision suggest three loops running at different speeds. The fast loop predicts, and may settle temporary states. The middle loop updates selected parameters after outcomes arrive. The

Algorithm 2 Concurrent serving, local learning, and structural discovery (proposed deployment)

-
- 1: Pin an active predictor version and its compatible memory interfaces
 - 2: **for** each incoming prefix **do**
 - 3: Read causal evidence; settle temporary state within a fixed budget
 - 4: Emit and record the pre-outcome prediction and version
 - 5: When the outcome arrives, append an immutable experience record
 - 6: Schedule bounded local updates, including retention data, on a learner copy
 - 7: **end for**
 - In background, independently of the fast serving loop:*
 - 8: Detect repeated residual patterns; propose and evaluate structural changes
 - 9: Train the guide on completed, correctly attributed search outcomes
 - 10: Test candidate versions on new-task, old-task and composition probes
 - 11: Publish an accepted version atomically; retain rollback and memory migrations
-

slow loop searches for new structure, consolidates what has been found, and updates the guide. To let these coexist, the system maintains an active, versioned predictor alongside a background learner. A new version is published in a single atomic step after it has passed its checks, and predictions already in flight continue with the version they started with. Algorithm 2 sketches the arrangement.

This schedule allows learning to proceed while inference continues, but it has not been benchmarked as an asynchronous service. Stale activities, readers that are shared and mutable, and incompatible memory keys can all break the intended semantics. PC’s local signals may make modular scheduling easier. They do not remove the need for consistency protocols.

7 The working cap: a concrete generalized-transformer instance

Two hybrid caps have been built. The first, **H1**, was a small neural cap whose results were instructive but negative, and it is described with those results in Section 8.6. This section describes its successor, **H2**, which is the working system behind the main results and the forgetting study. H2 is deliberately simple. It has a frozen base transformer, a bank of memory *readers* that each propose a full next-byte distribution, and a small trainable *gate* that decides how far to trust each reader in the current context.

7.1 Readers that report full distributions

The base is a frozen transformer with two blocks, width 64, four attention heads and a context of 64 bytes. Its vocabulary is the 256 byte values, and it has **120,576 distinct parameters**. The gate has 1,521 parameters. Neither count includes the reader dictionary described below, which is substantial.

The reader bank offers 13 possible readouts: the base model itself, eight suffix models differing in context depth and smoothing strength, and four readers of the copy family, one of which is a composed reader that falls back from copying to a suffix model. Appendix D lists the exact choices. Every reader supplies a complete distribution over the next byte, and all share the same uniform floor, so that no byte ever receives probability zero.

A *suffix reader* is the count-based predictor of Section 2. With recursive backoff, its prediction from a stored context s is

$$q_k(y \mid s) = \frac{n(s, y) + \alpha q_{k-1}(y \mid s')}{n(s) + \alpha}, \quad (30)$$

where $n(s, y)$ counts how often byte y followed s in the dictionary text, $n(s)$ is the total count for s ,

s' is the context shortened by one byte, and α sets how much weight the shorter context’s prediction receives. Contexts absent from the dictionary inherit the lower-order prediction unchanged. A *copy reader* works within the current 64-byte prefix alone. It finds earlier completed occurrences of the current suffix, counts the bytes that followed them, and blends those counts with a fallback distribution from the base or from a suffix reader. This is the mechanism of the captain example. It never reads the byte it is trying to predict.

The suffix dictionary is fitted on the first 700,000 bytes of the training text and then frozen. It holds 89,746 contexts and 388,416 next-byte count entries, and occupies about 21.97 MiB when serialized. This additional memory is central to H2’s gains, and the results should be read with it in mind. Restricting the cap to five active readers does not shrink the dictionary in proportion, because the readers share storage, and the present implementation computes the whole bank before masking the inactive readouts.

The decision to have readers report full distributions was the main change from H1. H1 summarized each reader in a fairly small table of learned features. H2 lets each reader say exactly which bytes it supports and with what probabilities, so the gate receives far richer evidence. It is the difference between a flag saying “something relevant was found” and a list of concrete candidate answers with their weights.

7.2 The gate: a learned prior adjusted by recent evidence

The gate has to turn the current context into a set of trust weights over the active readers. It does so in two stages: a learned linear map proposes weights, and a short settling process adjusts the proposal in the light of how well each reader has predicted the last few bytes.

The gate’s input is a feature vector $f(x) \in \mathbb{R}^{116}$, made up of the base model’s hidden vector (64 coordinates) and four summary features for each of the 13 readers: the entropy of its distribution, its largest probability, the support behind it, and its agreement with the base model’s most likely byte. Features are normalized with statistics fixed on the fitting set, and the features of inactive readers are masked. The linear gate proposes logits

$$u_\theta(x) = Wf(x) + b, \quad w(z) = \text{softmax}(z),$$

of which only the coordinates of active readers are used. Now take the last $m = 16$ bytes that have already been observed, and let q_{js} be the probability that reader j gave, before seeing it, to the byte that did occur at past position s . The gate’s energy is

$$E_0(z; x) = \frac{1}{2} \|z - u_\theta(x)\|^2 + \lambda \frac{1}{m} \sum_{s=1}^m \left[-\log \sum_{j \in \mathcal{A}} w_j(z) q_{js} \right]. \quad (31)$$

The first term keeps the trust logits z close to the learned proposal. The second rewards combinations of readers that would have explained the recent, already observed bytes well. Natural logarithms are used inside the energy, and reported losses are converted to bits. Settling this energy for K steps gives the free state z^K , and the prediction is the mixture of Equation (21) with weights $w(z^K)$.

During training, the new target y enters through the nudged energy

$$E_\beta(z; x, y) = E_0(z; x) + \beta \left[-\log \sum_j w_j(z) q_j(y | x) \right]. \quad (32)$$

The current target appears in this nudged phase and in scoring, and never in the free phase used for prediction. When we call the inference “causal” we mean only that it respects this boundary between what has and has not yet been observed. No claim about identifying causes is involved.

Algorithm 3 One H2 PC gate update**Require:** Prefix-only features f , past reader likelihoods Q , current target likelihoods q_y

- 1: $u \leftarrow Wf + b$; $z_f \leftarrow \text{SETTLE}(E_0, u, K)$
- 2: $z_c \leftarrow \text{SETTLE}(E_0, z_f, K)$
- 3: $z_n \leftarrow \text{SETTLE}(E_0 + \beta C(\cdot, q_y), z_f, K)$
- 4: $d \leftarrow (z_c - z_n)/\beta$
- 5: Form batch means of $df^\top$ and d ; clip; update W, b with AdamW
- 6: At prediction time, use only $\text{softmax}(z_f)$ and current reader distributions

The chosen configuration uses $\lambda = 2$, $\beta = 0.05$, 24 settling steps, settling step size 0.3, AdamW learning rate 0.003, weight decay 0.002, minibatches of 256 and gradient clipping at 1. These values were fixed during development, before the final test evaluation.

If the free state z_f and the nudged state z_β were computed exactly, the PC learning rule for the gate would take the local form

$$d = \frac{z_f - z_\beta}{\beta}, \quad \hat{\nabla}_W C = df(x)^\top, \quad \hat{\nabla}_b C = d, \quad (33)$$

with batch averages passed to the optimizer. The implementation settles for a finite number of steps, and it handles the resulting inexactness as follows. A free state is first settled for 24 steps. From that state, an unnudged control and a nudged state each continue for a further 24 steps, and the difference between these two replaces $z_f - z_\beta$. Any drift that remains because the free state had not fully converged affects both continuations alike and cancels. No gradient is backpropagated through this PC inference. The BP control differentiates instead through the same 24-step free inference that is used at prediction time. Algorithm 3 gives the update.

7.3 Choosing the structure: which readers to keep

Around this gate, H2 compares three strategies for choosing the structure. The *fixed* architecture uses five readers chosen in advance. *Forward* CD search starts from a small set and adds readers, ranking each candidate by the improvement it brings when inserted into the mixture and then refitting within a bounded budget. *Compression-first* CD search fits all 13 readers and then removes readers until five remain, refitting the candidate removals and their controls along the way.

Compression-first search consumes 1,056 optimizer updates in all, counting the trials that are rejected: 240 initial updates, then eight rounds of removal, each with two candidates and one control at 24 updates apiece, then 240 final updates. Each removal is selected under a constraint that forces the number of readers down, so it need not improve on the unconstrained full ensemble. The base reader always remains available. The fixed-graph controls that are matched to this search receive 1,056 ordinary updates. Equal numbers of updates do not amount to equal floating-point work.

A separate *transfer* condition takes the five-reader graph found in development and simply refits its gate. This tests the reuse of one search result, and is no evidence about a trained search guide generalizing across domains. The ensemble of all 13 readers serves as the reference for accuracy. H2 optimizes predictive quality under a budget on active readers. It does not compute the ideal sufficient quotient of Section 4, and it does not demonstrate that its code cost is repaid in full.

7.4 H2 as a generalized transformer

Each part of H2 corresponds to a GT operation. The suffix and copy lookups carry out source selection and transport. The reader distributions expose predictive distinctions. The hidden state and the reader statistics form the local product and its features. Equation (31) is the reconciliation step. The normalized mixture is the output re-encoding. Structural CD changes which reader coordinates are active, and PC or BP changes the gate parameters. Because the parts are separable in this way, each can be removed or replaced to measure its contribution, which is how the comparisons in the next section are made.

H2 is a restricted cap. The transformer and the dictionary are frozen, the grammar offers only 13 readers, and the only numerical plasticity is a small gate. These restrictions make the evidence cheap to obtain and straightforward to interpret. They also limit what can be concluded about a complete, standalone CD learner.

8 Experimental evidence

8.1 How to read these experiments

The experiments form a research sequence, and they should not be read as a single benchmark in which every system was held to identical conditions. The sequence began with finite program-discovery tasks on synthetic binary data. It continued through six successive text prototypes of CD, referred to below as v1 to v6, which began as standalone predictors and became caps on frozen base models. It then moved to the two hybrid caps, H1 and H2, and finally to a study of forgetting using H2. Along the way the roles assigned to data, the evaluation segments, the fitting budgets and the objective all changed, and H2 changed the evidence interface and added a large frozen dictionary of counts. Scores from different stages are therefore kept in separate tables, and only comparisons within one stage carry much weight. Table 3 summarizes the stages.

All text scores are next-byte cross entropies in bits per byte, and lower is better. “Gain” always means the comparator’s loss minus the new method’s loss, so a positive gain is an improvement. Repetitions with different random seeds generally share the same frozen base, corpus and dictionary, and sometimes the same search guide. They show that a result does not depend on one lucky run, and they are not independent replications with fresh pretraining or fresh data. Tiny Shakespeare was used repeatedly to guide development, as is reasonable in an exploratory phase, and the results carry that limitation. A longer accelerator run has been prepared as a screening step for the CD text prototypes. It has not been run, and no result from it appears here.

8.2 Finite program-discovery tasks

The first experiment used routed binary inputs and a grammar of candidate rules containing parity, majority and multiplexer functions. The system had to discover which rule generated each output. On the small tasks, search ordered by the history-trained guide reached an average accuracy of 97.20% on unseen outputs, against 83.06% for random ordering, and it needed 2.33 candidate trials per fit against 110.17. The time per fit was nevertheless slightly *higher* for guided search, 0.200 against 0.174 seconds, because the saving in candidates did not outweigh the overhead of the guide and of Python, and the cost of warming up the guide came on top of that. Both arms started from the same prior programs and history, so the comparison is about the ordering of search when motifs recur.

Several smaller demonstrations belong to this stage. An incomplete base model for an exclusive-or task improved from 50.05% to 98.05% accuracy when a cap found the missing parity residual. Finite code

Stage	Main evaluated object	Replication	Scope
Finite CD	Program/rule discovery and conditional cap	Small: 3 seeds; medium: 1	Synthetic binary motifs; exact code demonstrations
CD text v1–v5	Standalone finite CD and later graph caps	Pilot dependent	Objective and guide development; changing test blocks
CD text v6	Graph cap above three frozen base types	48 runs; 2 search seeds	History, precision, feature alignment, shadow compaction
Hybrid H1	Neural CD–PC/CD–BP cap	24 main runs; 3 seeds	Local learning plus structural growth and guide
Hybrid H2	Distribution-reader CD–PC/CD–BP cap	75 final runs; 5 seeds	Reader interface, pruning/reuse, settling and update rule
Forgetting	Continued H2 gate adaptation	35 runs; 5 seeds	Two independent domain shifts and replay controls

Table 3: The experimental stages, and why scores should not be compared across them. Development searches and auxiliary audits are additional to the counts of main runs.

packets were encoded and decoded exactly. A case in which the variable bindings were shifted could not be solved within budget. And when observers were restricted to pairs of inputs, a three-input parity relation could not be recovered, as the argument of Section 4.6 predicts. These results support the value of selected high-order tests and of reusable search motifs. They do not show unrestricted creative discovery.

8.3 Early text prototypes

The first text prototype, v1, showed that a standalone finite CD predictor could operate autoregressively on real text. Guided CD scored 3.3132 bpb, better than a bigram model’s 3.5862, while an interpolated 4-gram model scored 2.6523. The transformer base in this pilot had been trained for only 300 steps and scored 3.8454. Both a CD cap and a conventional neural cap improved on it. With a base that weak the result could say little about caps on a competent model, and it prompted a control with a stronger base.

The second prototype, v2, trained the transformer for 2,500 steps. Its CD cap selected no additions at all and stayed at the base score of 2.894651 bpb, while a conventional neural cap reached 2.888592. A CD cap on the 4-gram base also selected nothing, and historical guidance did worse than simpler choices. The mechanism ran correctly. What failed was the admission rule: the strict short-horizon model-cost criterion discussed in Section 4.4 filtered out corrections that might have helped.

8.4 A revised admission criterion, and further development

The third prototype, v3, replaced that admission gate by predictive selection under explicit resource budgets, as in Equation (15). It also broadened the warm-up of the guide, allowed bundles of proposals, and kept an MDL-gated arm as a control. Table 4 gives two-seed means on this stage’s own final block of 4,096 contiguous bytes. The columns compare the frozen base with caps whose candidates were ordered by the history-trained guide, ordered by a *proxy* (a cheap score computed from the current residual, with no learned history), or admitted through the old MDL gate, and with a conventional neural cap.

The caps freed from the MDL gate improved all three bases in both seeds, which is useful evidence for the revised objective. Compositions selected by the search were kept, although controls with no composition could still be competitive. The accounting of Equation (14) remains sobering. One pool of modules for the transformer base occupied 84,975 bytes and saved 0.034199 bpb, which implies about

Frozen base	Base	Guided CD mixture	Proxy mixture	MDL-gated mixture	Neural cap
Bigram	3.522291	3.137575	3.146906	3.311762	3.196583
Transformer	2.849888	2.816860	2.834088	2.849888	2.846757
4-gram	2.343317	2.332640	2.343561	2.343317	2.343549

Table 4: CD text v3, two-seed mean bpb. The scored block differs from the one used for H1 and H2. The neural caps are matched to the CD caps in the number of optimizer target exposures during deployment, excluding warm-up, and not in total computation.

19.9 million future bytes before the storage is repaid at an unchanged gain, with warm-up and running time as further costs.

The fourth prototype, v4, added several mechanisms at once: joint selection of the mixture, nested protection of earlier predictors, progressive consolidation of several experts into smaller ones, exact caching and continuation of graphs, and richer guidance. The changes helped in some settings and hurt in others, and the failures could be traced to particular causes. A guide with several outputs on different scales devoted most of its capacity to predicting cost and little to predicting gain. A curriculum of short fits changed the optimization it was supposed to approximate. And replacing a teacher model by its compact student could alter the course of the subsequent search. These were concrete problems of implementation and objective. The general lesson was that a larger number of mechanisms is no improvement in itself.

The fifth prototype, v5, separated the prediction of utility from the prediction of cost, and introduced the *shadow*: a compact student distilled from a teacher and kept alongside the teacher, as an additional predictor and as a possible starting point for later search, with the teacher retained. In v5’s comparison over two seeds and three bases, fitting experts individually beat mixture-aware fitting in all six base–seed pairs, and the transformer mean improved from 2.835960 to 2.823919 bpb on that stage’s 16,384-byte continuation. More historical data helped in selected audits, but richer corrections were often calibrated to zero after partial fitting. It also emerged that the guide had been trained on features computed from samples of one size and deployed on features computed from samples of another. That mismatch motivated v6.

8.5 CD text v6: useful mechanisms, uneven outcomes

The sixth prototype completed 48 deployment runs (eight variants, three frozen bases, two search seeds) and 9,216 measured candidate trials for training and auditing the guide, spread across independent warm-up states, real search trajectories, calibration and audit. Its ingredients were feature samples of matching size in warm-up and deployment, individual fitting of experts, continuous mixture weights, signed outcomes relative to matched controls, calibrated tower descriptors, more precise outcome labels, online updates of the guide, and budgeted trials of compact shadows.

The rows of Table 5 need some explanation. “Saved v5 individual” is the best v5 configuration, rescored on this segment. The two “proxy” rows order candidates by the cheap residual score without a learned guide, using either mismatched (“coarse”) or matched (“aligned”) feature samples. The “history” rows use the learned guide. It is trained either on independently constructed search states or on the successive states of real search trajectories, and with either a simple or a rich set of guide features. The last three rows each change one thing relative to the trajectory-rich arm: outcome labels scored on more targets, a guide updated online during deployment, or the addition of compact shadows. Every score is that of the newly learned branch alone, on an additional segment of 12,288 bytes that excludes bytes seen in earlier evaluations.

Variant	Bigram	N-gram	Transformer
Frozen base	3.616049	2.702702	2.940145
Saved v5 individual	3.233547	2.700138	2.911783
Coarse proxy	3.204889	2.702254	2.913690
Aligned proxy	3.177026	2.702780	2.912466
Independent simple history	3.179134	2.697915	2.909081
Trajectory simple history	3.179664	2.699798	2.913567
Trajectory rich history	3.181327	2.698587	2.912926
More precise outcome labels	3.178098	2.699127	2.907259
Online guide updates	3.183215	2.697962	2.912040
Compact shadow trials	3.188837	2.699130	2.905906

Table 5: CD text v6, two-seed mean bpb on an additional 12,288 bytes. Every row except the frozen base reports the newly learned branch. The score of the protected outer mixture is never substituted for it.

Four effects within v6 were positive in both search seeds and on both reported segments:

- Aligned feature samples, compared with coarse ones, improved the bigram cap by 0.027863 bpb.
- A simple guide trained on independent states improved the n-gram cap over the aligned proxy by 0.004865 bpb.
- Outcome labels measured on 32k targets, compared with 8k, improved the transformer cap over its trajectory-rich control by 0.005667 bpb.
- Budgeted shadow distillation improved the transformer cap over the trajectory-rich arm without shadows by 0.007021 bpb.

Two qualifications apply to these comparisons. The n-gram history arm used more of its permitted ceiling of optimizer tokens than the proxy did, so the two arms did not consume identical compute. The shadow arm used the same 595,968 optimizer tokens per seed as its control without shadows, but its graphs and retained predictors differ in size, which changes floating-point work and storage.

There was no general victory for v6. The transformer shadow arm beat the saved v5 branch by 0.005877 bpb on the additional segment and lost to it by 0.009763 on the previously evaluated segment. Rich guide features showed no robust benefit of their own when the influence of the guide was held fixed in audits where every candidate was probed. Histories drawn only from trajectories did not consistently beat varied independent-state histories, and one reason is easy to see: the n-gram trajectory sample contained only 42 distinct incumbent states among its 1,024 labels.

Two lessons for guide design follow. The guide needed better experiences and more reliable measurements far more than it needed additional input features, since a thousand trials made in a few similar situations can teach less than a smaller number spread over different situations. And a compact student can help later search by making room to grow, even when it does not immediately beat its teacher.

The deployed v6 predictor wraps each newly learned branch in a protected mixture with an older branch, in the manner of Equation (36), so that its cumulative loss cannot fall far behind the older one. Such a safeguard could hide weak learning in the new branch, which is why the tables report the new branch by itself. The implementation also passed causal checks, audits of the optimality of its mixture weights, and round trips through an arithmetic coder. Appendix F summarizes the scope of that validation.

H1 condition	Blended bpb	Raw bpb	Seconds	Updates
Fixed BP	2.874994	2.883675	1.57	408
Fixed PC	2.875953	2.885187	5.65	408
CD-BP	2.877670	2.882826	8.70	1560
CD-PC	2.877332	2.883586	29.02	1560
CD-PC, no history	2.877265	2.883600	27.94	1560
CD-PC, random search	2.877523	2.884566	27.31	1560
Long fixed BP, round checks	2.876735	2.888991	5.54	1560
Long fixed PC, round checks	2.882814	2.885655	21.50	1560

Table 6: Hybrid H1, three-seed means on 32,768 identical test targets. The frozen base scores 2.896241 bpb. Times exclude shared caching, warm-up histories and final evaluation. The two “long” controls evaluated checkpoints only at the ends of rounds; an audit with denser checkpoints is described in the text.

8.6 Hybrid H1: a useful negative result

H1 was the first cap to combine CD structure search with PC learning. It attached a nonlinear cap to the same small frozen transformer later used by H2. The cap had 32,064 allocated parameters, a finite grammar of observers, parameter fitting by either PC or BP, growth of structure by CD, screening of candidates by residuals, and a small history guide. The main test scores use a blend of base and cap whose weight was selected on development data. Table 6 also shows the raw cap scores, because the blending changed the ranking of PC against BP.

The simplest condition did best. The fixed BP cap beat CD-PC in every one of the three seeds. CD-PC beat CD-BP by only 0.000338 bpb on average after blending, and the order of the two reversed for the raw caps. Historical guidance had essentially no effect on deployed performance and slightly worsened an audit of structural choices under PC. Diagnostics in which a reader’s features were erased while the rest of the cap stayed frozen did show that the readers added by search contributed to prediction, but not that the grown architecture had any advantage over a fixed architecture trained afresh.

One thing H1 did establish is that the PC machinery computes what it is meant to compute. Its updates approached BP gradients as the nudge was made smaller and settling longer, as Theorem 6.1 predicts. At that stage PC offered no capability that was specific to it and useful. This pointed to two changes for the next cap: a richer evidence interface, and a form of prediction-time inference that does some useful work of its own, since additional PC sweeps alone had achieved little.

A later audit evaluated the long fixed controls every 48 updates. Table 6 had evaluated them only at round boundaries. With the same training budget of 1,560 updates, and with the checkpoint selected on validation data, the test means were 2.878186 bpb for BP and 2.877808 for PC, and every seed selected update 168. This audit of checkpoint policy likewise found no consistent advantage for structural PC learning.

8.7 Hybrid H2: richer readers, explicit inference, and compression

H2 is the system described in Section 7. The base and the dictionary are frozen. The gate is fitted on 16,384 training targets lying outside the prefix that was used to build the dictionary. The choice of configuration was kept separate from the choice of checkpoint and graph, with different validation data for each. Final scores are reported on two segments: the original test segment of 32,768 targets, and the 22,938 targets that follow it. Twelve development models came before 75 final models whose recipes

H2 condition	Original test	Following segment	Updates
Frozen transformer	2.896241	2.963721	–
Global all-reader mixture	2.428546	2.506746	EM
Fixed five, BP	2.356488	2.422106	240
Fixed five, PC	2.356485	2.421712	240
All 13, PC	2.332323	2.403255	672
Reused graph, PC	2.341418	2.414385	240
Reused graph, no inference	2.343121	2.415047	240
Compression-first CD–BP	2.334918	2.409146	1056
Compression-first CD–PC	2.335741	2.410003	1056
Fixed five, PC, matched	2.345958	2.415914	1056
Reused graph, PC, matched	2.337033	2.410459	1056

Table 7: Hybrid H2, five-seed mean bpb. The “global” mixture uses a single set of reader weights for all contexts, fitted by expectation–maximization (EM). “Matched” controls receive the same 1,056 updates as compression-first search. Conclusions should be drawn from the controls within this table. The large gain over H1 also reflects the added dictionary and the new evidence interface.

were locked in advance: 50 in the main comparison, 10 for compression-first search, and 15 controls with matched update counts. All final conditions use the five seeds 7, 19, 37, 53 and 71. Table 7 gives the principal conditions, and Table 12 in the appendix gives all fifteen.

Compression-first CD–PC reaches 2.335741 bpb on the original segment. The base scores 2.896241 on the same targets, and H1’s CD–PC scored 2.877332. This is a substantial improvement in prediction. It cannot be credited to the PC update rule or to structural compression alone, because the dictionary and the full-distribution readers constitute a major change in architecture and in the use of data. The controlled comparisons within H2 are more informative about what contributes what.

- A single global mixture of all the readers, with no dependence on context, already reaches 2.428546 bpb. The richer evidence is therefore useful before any gate has learned anything.
- Compression-first CD–PC beats the fixed five-reader PC control given the same 1,056 updates by 0.010217 bpb on the original segment and by 0.005911 on the following segment, in all five seeds on both. Choosing the structure by search is worth something.
- The full 13-reader PC ensemble remains better than the compressed five-reader PC cap, by 0.003418 and 0.006749 bpb on the two segments. Compression gives up a small amount of prediction in exchange for fewer active readouts. The reference implementation does not demonstrate an end-to-end saving in time or storage from this.
- Compressed CD–BP is slightly better than compressed CD–PC, by 0.000823 and 0.000858 bpb. There is no basis here for claiming that the PC learning rule is superior.
- Reusing the graph found in development improves on the predefined five-reader PC graph by 0.015067 and 0.007328 bpb. This is useful reuse of structure within one corpus. It was not produced by a trained history guide.

8.8 Does settling at prediction time add anything?

Two experiments address whether prediction-time inference contributes. In the first, a trained compressed PC gate is held fixed and settling is switched off at prediction time. The loss on the original segment worsens by 0.002484 bpb, in all five seeds. If settling is left on but each example is given the recent-

evidence record of a different example, which is a deliberately invalid input serving as a negative control, the loss worsens by 0.002305 bpb. So the benefit of settling comes from the relevant recent evidence. It does not come from the extra computation as such. On the following segment, switching settling off costs 0.001070 bpb.

In the second experiment, gates are retrained on the reused graph with and without inference. Settling improves the test score by 0.001703 bpb on the original segment and by 0.000663 on the following segment, again in all five seeds. The contribution of prediction-time inference is small and consistent. It should be kept distinct from the much larger effect of the reader dictionary.

Audits of H2’s gradients show close agreement between PC and BP at the chosen settings, and tighter agreement closer to equilibrium. Seven mathematical, causal and checkpoint tests passed. The 450 saved loss means were recomputed independently, and the fingerprints of all 75 checkpoints and their sources stayed fixed through evaluation. These checks support the implementation and the stated comparisons. They say nothing about behavior at scale or about resistance to forgetting.

8.9 What the experiments support so far

The evidence favors a strong and explicit reader interface, the evaluation of structural choices by their measured effect on the deployed predictor, keeping useful teachers when students are distilled from them, a history-guided search whose outcome labels are calibrated, and a modest amount of prediction-time inference over recent evidence. It equally favors keeping simple controls in every comparison: ordinary numerical fitting, screening by residuals alone, the full reader bank, and direct reuse of a graph.

The evidence does not establish that rich tower features always help, that PC is a better parameter learner than BP, that more search always improves prediction, or that a standalone CD network can compete with a large, well-trained transformer. The best empirical result to date is a small memory-assisted cap. Its value for the program is as an intermediate target whose components can each be tested independently.

9 Forgetting: what happens when the cap learns something new?

Continual learning is the first goal of Section 1, and H2 allows a direct if small test of it. We take a cap trained on Shakespeare, continue training it on a different kind of text, and measure two things: how much it learns about the new text, and how much worse it becomes on Shakespeare.

9.1 Protocol, and what is meant by forgetting

Training was continued from the five H2 checkpoints produced by compression-first PC search. Each was adapted independently to two new domains, *Alice in Wonderland* and Python source code. For every domain and seed, three arms each received 1,024 updates: PC, BP, and PC with 10% replay. Five further PC runs continued training on Shakespeare alone, as a control for the effect of simply running the optimizer longer. That makes 35 adaptation and control runs. Along their trajectories 595 checkpoints were saved, and all were hashed before any held-out evaluation took place.

The base, the reader dictionary, the selected graph of readers and the feature normalization all stayed frozen. Only the gate was trainable, and it continued from its original checkpoint together with the saved state of its AdamW optimizer. PC and BP received the same minibatches of new-domain data, and both used the same inference at prediction time. In the replay arm, 26 of the 256 examples in each minibatch were replaced by samples from a fixed buffer of 1,024 examples of old *training* data. This is 10.16% replay, and it means slightly less exposure to new data for the same number of updates.

New domain	Update	Old loss after	F_A	Advantage erased	G_B
Alice	PC	2.401712	+0.065971	11.8%	+0.087650
Alice	BP	2.401913	+0.066171	11.8%	+0.087625
Alice	PC + replay	2.374663	+0.038921	6.9%	+0.085625
Python	PC	2.414782	+0.079041	14.1%	+0.251409
Python	BP	2.415367	+0.079625	14.2%	+0.251357
Python	PC + replay	2.385189	+0.049448	8.8%	+0.249538

Table 8: Forgetting after 1,024 updates, five-seed means in bpb. The new-domain losses before adaptation were 2.915507 (Alice) and 4.354498 (Python). A positive F_A is deterioration on the old domain, and a positive G_B is successful learning of the new one.

The Alice text was divided into disjoint contiguous regions for training, validation and testing. The Python data used nine modules for training, one for validation and six different ones for testing. Each new domain supplied 16,384 training targets, 4,096 validation targets and 16,384 test targets, and no context crossed the boundary of a split or a module. Retention on the old domain was scored on the two Shakespeare test segments already used for H2. Because those segments had been used before, this is exploratory evidence about retention. It is not a benchmark on an independent corpus.

Let θ_A be the original checkpoint and $\theta_{A \rightarrow B}$ the checkpoint after adaptation to the new domain B , and write L_A and L_B for the losses on the old and new domains. Define

$$F_A = L_A(\theta_{A \rightarrow B}) - L_A(\theta_A), \quad G_B = L_B(\theta_A) - L_B(\theta_{A \rightarrow B}). \quad (34)$$

A positive F_A is forgetting and a positive G_B is learning of the new domain. A frozen model has both equal to zero, which is why both have to be reported. To give forgetting a scale, we also report

$$E_A = \frac{F_A}{L_A(\text{frozen base}) - L_A(\theta_A)}, \quad (35)$$

the fraction of the cap’s original advantage over the frozen base that adaptation has erased. This is a convenient yardstick for the present experiment. We do not offer it as a universal definition of catastrophic forgetting.

9.2 Results: consistent interference, with most capability retained

Before adaptation, the mean Shakespeare loss was 2.335741 bpb, against 2.896241 for the frozen base. Table 8 gives the state of affairs after the full 1,024 updates.

Both forgetting and new-domain learning occurred in every seed of both PC transfers without replay. After adapting to Alice the cap kept about 88.2% of its original advantage on Shakespeare, and after Python about 85.9%. Perplexity on the old task rose by factors of about 1.047 and 1.056. *Moderate forgetting* is a fair description of effects of this size. They fall well short of catastrophic collapse, and they also show plainly that this prototype is not immune.

Replay reduced mean forgetting by 41.0% on Alice and by 37.4% on Python, while keeping 97.7% and 99.3% of the gain on the new task. The control that continued training on Shakespeare changed the original loss by only -0.001287 bpb on average, and by $+0.000559$ on the following segment. The losses in Table 8 are therefore due to interference from the new domain and cannot be put down to the optimizer merely running for longer. Figure 3 shows how forgetting and new learning develop over the course of adaptation.

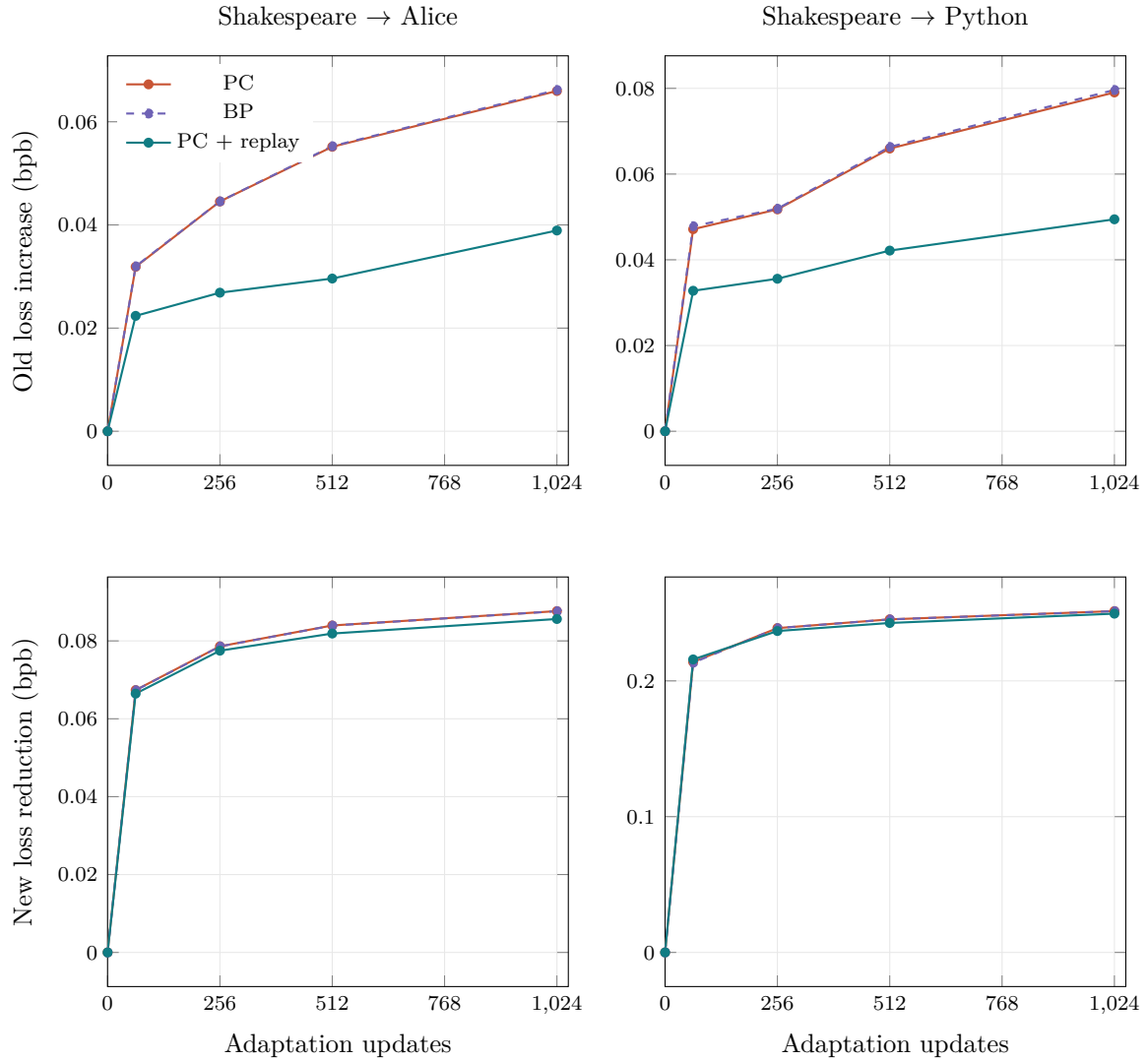


Figure 3: Mean adaptation trajectories over five seeds. The upper panels show forgetting and the lower panels show new learning. The PC and BP curves almost coincide. Replay improves retention substantially and leaves most of the new-task gain intact. The model, the dictionary and the evaluation streams are shared across seeds.

The picture does not depend on stopping at the final update. If the checkpoint is chosen instead by validation loss on the new domain alone, PC forgetting is 0.059998 bpb for Alice and 0.076247 for Python, compared with 0.034942 and 0.049853 with replay. On the following Shakespeare segment, final PC forgetting without replay is 0.074235 and 0.080161 bpb, and replay reduces these to 0.043715 and 0.051220. Appendix E.3 gives the ranges across seeds and descriptive block intervals.

9.3 Why frozen readers can coexist with forgetting

It may seem odd that a system whose memory is frozen forgets anything. The explanation lies in Equation (21). The readers q_j stay fixed, but the weights $w_j(x; \theta)$ that decide how far each is trusted do change. An update on the new domain can leave every stored count untouched and still change how the old evidence is used. The present system retains the *availability* of its knowledge more firmly than it retains *effective access* to it, which is the distinction drawn under the first goal in Section 1.

An analogy may help. The library still has all the old books. The librarian, however, has picked up new habits about which shelf to consult, and now sends an old question to the wrong shelf more often than before. Glancing at how recent questions turned out helps the librarian reconsider, which is the role settling plays. Replay gives the librarian practice at answering old questions while learning the new job.

Settling at prediction time does provide some protection, without removing the effect. With the same frozen checkpoints from before and after adaptation, switching settling off raises the estimated forgetting from 0.065971 to 0.070766 bpb for Alice and from 0.079041 to 0.083149 for Python. This diagnostic changes inference only, and is a different thing from retraining a model without PC. As learning rules, PC and BP behave almost identically throughout the main adaptation comparison.

The appropriate engineering response is explicit protection of retention: replay, probes that guard the routing of protected examples, separate retained gates or branches, growth of modules under version control, and cautious consolidation. Of these only replay has been measured. The others are hypotheses, and they call for the same paired tests of retention and plasticity.

9.4 What can be guaranteed about retention

Some retention guarantees are available, and it helps to be clear about what they cover. They concern complete predictors that are kept and protected. A frozen component inside a predictor that continues to change enjoys no such guarantee.

Suppose the deployed predictor is a fixed mixture of a retained old predictor and a new one,

$$p_{\text{safe}}(y | x) = \rho p_{\text{old}}(y | x) + (1 - \rho) p_{\text{new}}(y | x), \quad \rho > 0.$$

Then on every single target, its log loss exceeds that of the old predictor by at most $-\log_2 \rho$ bits. This bounds the deterioration and does not make it zero, and if ρ is small the bound is loose enough to permit substantial forgetting.

A Bayesian mixture over whole sequences gives a different, cumulative guarantee. For retained predictors P_j , each evaluated causally, let $P(x_{1:T}) = \sum_j \pi_j P_j(x_{1:T})$ with prior weights $\pi_j > 0$. Then

$$-\log_2 P(x_{1:T}) \leq -\log_2 P_j(x_{1:T}) - \log_2 \pi_j. \quad (36)$$

This follows at once from $P \geq \pi_j P_j$, and the corresponding online weights are updated only after each outcome is revealed. The guarantee is that the cumulative code length is never more than a fixed number of bits worse than that of any retained branch. It does not promise good performance on every sub-block, successful routing between tasks, preservation of generated trajectories, or free storage of the retained

models, and each time the mixture is restarted the prior overhead is paid again. H2’s learned gate is not a protected Bayesian mixture of this kind, so Equation (36) cannot be invoked for the forgetting experiment above.

9.5 The continual-learning question that remains

The present test covers a small gate over frozen evidence and a fixed structure, two independent transfers, and a finite number of updates. It does not test a long stream of tasks, updates to the reader memory, deletion of old modules, or joint training of the base. A stronger study would combine repeated returns to earlier domains, a fixed total memory budget, selective growth and retirement of modules, and comparisons among replay, retained gates and regularization. It would have to measure new learning as carefully as retention, so that a conservative model cannot appear to succeed merely by refusing to change.

10 Scaling from the sandbox toward frontier systems

10.1 What survives approximation, and what does not

The ideal operations of Sections 3–6 specify what a useful computation should accomplish, and several of them cannot be afforded as algorithms. Computing an exact sufficient quotient can require answering an enormous family of future questions. Joint dependency tables grow exponentially with the number of variables. Program search branches combinatorially. Refitting everything globally can cost more than the prediction it improves is worth. And retaining every useful model eventually exceeds any finite memory.

The practical program is to hold on to a small number of measurable capabilities while replacing each expensive ideal by a controlled approximation. Table 9 pairs each ideal with a scalable approximation and with the check that the approximation makes necessary. The cap route is the nearer-term target, because a strong base supplies broad competence in language, the read and write interfaces can be fixed, and many candidate trials can share the base’s computations. One caution applies. A stronger base leaves smaller and noisier residual gains for a cap to find, so success on a weak base is not enough.

10.2 Stages of data and compute, with decision criteria

Table 10 proposes six stages. The ranges in it are bands for planning. They are neither measured scaling laws nor promises about running time. The later stages may use a subword tokenizer where the sandbox uses bytes, so models should be compared only within a common tokenizer, and bits per byte should be reported only when computed consistently.

Ideal operation	Scalable approximation	What must be checked
Preserve all future distinctions	Finite continuation probes, replay, protected branches	Coverage and worst observed regressions; no universal guarantee
Exact quotient or shortest program	Restricted grammar, local rewrites, shadow distillation	Lost behavior, actual code/storage cost, future search effects
Complete dependency tower	Selected scopes, sparse high-order motifs, sketches	Ability to recover useful high-order relations missed by low-order summaries
Exhaustive candidate search	Retrieved templates, learned proposals, exploration, bounded reservoir	Utility per total work; diversity; unseen-template tests
Global numerical refit	Local modules, low-rank writers, bounded PC/BP steps	Matched-control gain and optimization fidelity
Exact equilibrium at every token	Warm starts, early exit, amortized inference, selective settling	Prediction gap and retention relative to longer inference
All readers, full vocabulary	Sparse activation, shared fallbacks, normalized sparse corrections	Omitted mass, retrieval recall, vocabulary calibration
Unlimited model retention	External archive, small active pool, tested retirement	Access cost, routing retention and recovery of rare capabilities

Table 9: Approximations and the obligations that come with them. A cheap heuristic score does not become an exact bound by being called a certificate.

Table 10: A staged scaling program. The data quantities refer to additional exposure for adaptation and evaluation. They do not imply pretraining a competitive base from scratch.

Stage	Substrate and data scale	Principal experiment	Evidence required before expanding
A: corrected small studies	Existing 0.12M-parameter base; several fresh small corpora and longer domain streams	Replicate H2; add growing readers, replay, retained gates and strict memory ceilings	Consistent gains beyond reused Shakespeare; useful learning with controlled forgetting
B: single-accelerator cap	Approximately 10M–100M-parameter base; 10^7 – 10^8 additional tokens	Vectorized readers, sparse output path, mixed independent/trajectory histories, measured throughput	Gain over a strong ordinary adapter at matched total memory and compute; reusable guide signal
C: substantial open base	Approximately 0.1B–3B parameters; 10^8 – 10^9 additional tokens or task-equivalent episodes	Richer multi-layer read ports; composable cap modules; diverse retrieval and symbolic tasks	Stable interfaces, bounded latency, retention across many tasks, benefit from cap composition
D: distributed module ecosystem	Roughly 3B–30B base or a heterogeneous collection; multiple data owners/workers	Independent local discovery; exchange modules and guide examples; local validation	Positive transfer per communicated byte; robust non-IID and stale-version behavior

Stage	Substrate and data scale	Principal experiment	Evidence required before expanding
E: frontier-base cap	A frontier-class accessible base; evaluation volume set by measured residual effect size	Sparse recurrent workspace, external program/evidence memory, jointly prepared read ports	Reproducible capability or continual-learning advantage with deployment-level resource accounting
F: standalone GT/CD learner	Scale increased only after controlled smaller successes	Learn representations, readers, structure and numerical weights jointly from raw data	Competitive scaling curve and quality-per-total-compute against a well-trained transformer

Stages E and F answer different questions. A useful cap on a frontier base would not show that CD can replace the pretraining of that base. Conversely, a cap can have practical value even if the best substrate remains a transformer indefinitely.

10.3 A budget equation

Whether structural search can be afforded at scale comes down to simple accounting. Let C_B be the ordinary cost of the base per token, C_C the cost of the cap’s fast path, K the number of candidate trials in one search, C_T the cost of one trial, B the average number of tokens between searches, and C_H the total background work over T tokens. With everything in comparable resource units, the average cost per token is

$$\bar{C} = C_B + C_C + \frac{KC_T}{B} + \frac{C_H}{T}. \quad (37)$$

Holding the overhead to a fraction ρ of the base cost requires

$$C_C + \frac{KC_T}{B} + \frac{C_H}{T} \leq \rho C_B. \quad (38)$$

For illustration, suppose the fast path costs $C_C = 0.02C_B$, one search costs $KC_T = 2000C_B$, background work averages $0.01C_B$ per token, and the ceiling on overhead is $0.10C_B$. Then $B \geq 2000/0.07 \approx 28,572$ tokens must pass between searches. This is dimensional accounting and does not describe a measured operating point. It does show why structural search should usually be an occasional investment whose results are reused. Search at every token could not be afforded.

The same equation makes the hypotheses about learning from history specific. Experience could lower the number of candidates K that must be tried, lower the cost C_T of each refit through better initialization, lengthen the interval B over which a result is reused, or find better modules at the same expense. All four should be measured. Training a large guide in order to save a tiny amount of candidate work can be a poor exchange even when the guide’s ranking accuracy improves.

Accelerator time, CPU time, memory, energy and monetary cost should be reported as separate measurements unless an explicit conversion between them is given. Counts of optimizer tokens are useful within one design, but they are not floating-point operations. The costs of generating a guide from a cold start, of repeated passes through the base, of failed candidates, of serialization and retrieval, and of retained teachers all belong in the accounts.

10.4 The vocabulary and reader bottleneck

H2 can afford to materialize 13 distributions over 256 bytes at every position. With the vocabulary of a large language model in place of 256 bytes, the same naive approach becomes costly. Three changes are plausible.

1. Readers return sparse distributions, or sparse deviations from a shared fallback, and only a few readers are activated at each position.
2. Shared context statistics are cached, and compatible reader operations are batched on an accelerator, with no Python loops over tokens or graph nodes.
3. The base’s exact probabilities are reused for every vocabulary item a correction leaves untouched, and normalizers are computed without discarding the tail of the distribution.

The third point can be made exact. If the correction score $s(y)$ of Equation (19) is zero outside a finite set S of tokens, the normalizer is

$$Z = 1 + \sum_{y \in S} p_B(y \mid x)(2^{s(y)} - 1), \quad (39)$$

and every token outside S keeps its base probability divided by Z . This algebra avoids the common pretense that the probabilities of the top k tokens sum to one. Obtaining the base probabilities, and producing or sampling the final output, still have their costs. Approximate versions of the entropy, confidence and retrieval features may also be useful, but their accuracy has to be checked, because errors in them can change the routing.

Sparse mixture-of-experts systems are an engineering precedent for activating only part of a large model [14]. They do not show that irregular symbolic search will make efficient use of the hardware. CD modules should compile, wherever possible, to a small set of batched kernels, bounded interpreters or indexed operations, and slow exceptional programs can run off the fast path.

10.5 Predictive coding at scale: keep the useful inference, and charge for the settling

The experiments show a small benefit from reconciling the trust in readers with recently observed evidence. They do not show a cheaper learning rule, and in H1’s CPU reference implementation PC learning was much slower than BP. A scaling study should therefore keep three questions separate: whether activity inference improves quality, whether PC learning enables useful modularity or retention, and whether either repays its cost.

Practical approximations include good warm starts for the activities, a small learned predictor of the settled state, stopping when the residual falls below a tolerance, settling only on uncertain cases, and updating only a small active set of modules. The learned predictor can be distilled from a slower teacher, with the difficult cases sent back to explicit inference. When the energies are nonconvex some criterion of stability is needed, since descending the energy does not certify that the resulting prediction is globally good.

No method is cheapest as a matter of course. BP through unrolled inference pays in memory for stored activations. Implicit gradients need assumptions about the solver. PC needs extra settling and local state. Before a result is attributed to the learning rule, the methods compared should have the same access to readers, the same memory and the same prediction-time inference.

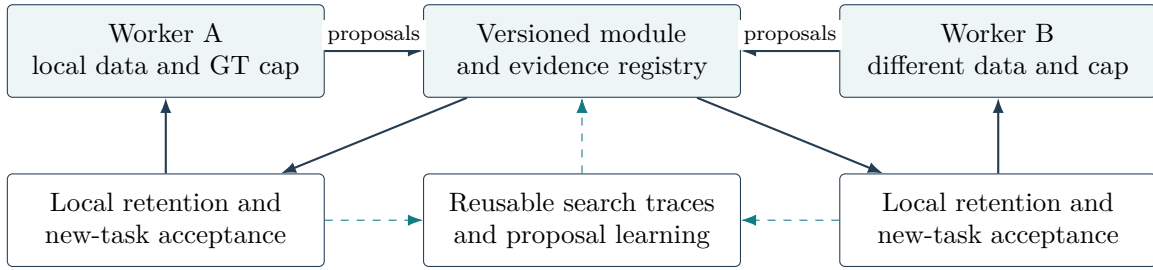


Figure 4: A proposed arrangement for decentralized learning. The registry is a logical interface for exchange and need not have a central owner. Receiving a module is a separate step from accepting it into a worker’s active predictor.

10.6 Decentralization as the exchange of modules

Local gradient rules are sometimes presented as the route to decentralized learning. The proposal here is different: what the learners exchange is modules. A portable module record might contain

$\mathbf{m} = (\text{typed ports, program/weights, version, dependencies, evidence, uncertainty, resource profile}).$

A worker publishes a candidate module together with examples from its search. A worker that receives it checks the compatibility of its interface, checks whether its information duplicates what the recipient already has, measures the gain on its own new tasks, and measures any regression on its old ones (Figure 4). Evidence summaries must record which examples or sources contributed to them, so that copied evidence is not later mistaken for independent support. Workers whose data differ may quite rationally keep different modules.

The quantities to measure are predictive transfer per communicated byte, the time taken to reach useful local adaptation, the number of duplicate discoveries avoided, and the damage done by stale or incompatible modules. A common shared embedding would be convenient, though symbolic ports and small adapters may allow bases of different kinds to take part. Exact semantic equivalence of arbitrary programs cannot in general be decided, and typed contracts with bounded probes are the practical substitute. No decentralized experiment has yet been run in this project.

10.7 Linking to symbolic systems, and testing creativity

The GT interface gives other AI systems a concrete point of entry. A PLN query could return a distribution together with its supporting evidence. A MeTTa program could compute a reader. A planner could propose a continuation. An episodic store could return a case with a record of its origin. The gate, or a shared workspace, then decides how each piece of evidence affects the prediction. Typed ports should distinguish among a fact, a probabilistic belief, an executable rule and a proposed hypothesis.

To test creativity, one should hold out whole *families* of problems, going beyond new rows of familiar text. The tests should separate new bindings, new combinations of known operators, new depths of dependency, and libraries of operators that have truly been extended. Generated rules should be validated by execution or by external evidence, and then tested for reuse on separate problems. CD search should be compared with a strong conventional model given the same external tools and the same compute at test time, so that mere access to more tools is not mistaken for an architectural advantage.

One intermediate experiment would be especially informative. It would combine text with explicit relational episodes: assignments of roles, updates over time, exceptions and counterfactual questions. The same facts can be presented in different wording, while the answer requires a new composition of

them. One would measure prediction, the recovery of executable rules, the cost of search, and later transfer. This connects the binding example of Section 3.7 to a tractable test of a neural-symbolic capability, and avoids the pretense that a lower loss on Shakespeare measures creativity.

10.8 What would justify a frontier-scale claim?

A cap would earn such a claim by showing reliable gains on strong bases, across independent domains, across streams of old and new tasks, and across combinations of modules, with memory and latency fully accounted for. A standalone replacement for the transformer would need, in addition, a competitive scaling curve for learning representations from raw data. Several model sizes and data budgets, at the least, are needed to distinguish a fixed advantage at small scale from a trend that is sustained.

The most promising working hypothesis is that dense learned representations remain an efficient substrate, while explicit reusable readers, local activity inference and history-guided structural search take over some forms of adaptation and reasoning. Whether the gains would eventually justify replacing substantial parts of the dense network is an empirical question. The present evidence makes progressively stronger caps a concrete way of investigating it.

11 Assessment and research priorities

The generalized transformer provides a common geometric language for transformer blocks, executable distinctions, memory readers and iterative inference. CD adds a principled place for structural discovery, refitting, compression and accumulated search experience. PC and BP are two ways of learning the numerical parameters inside that architecture. Caps make the whole program experimentally accessible, by retaining a competent base and narrowing what the new mechanism has to learn.

The experiments indicate a credible immediate direction. Readers should keep their full predictive content. Structure should be judged by its measured contribution to the deployed predictor. Strong fixed-graph and full-bank controls should accompany every comparison. And the learned routing policy should be protected during adaptation. Guidance by search history remains promising, particularly where motifs recur and outcome labels are precise, but added complexity in the guide should have to earn its place on independent trajectories. The observed benefit of PC, so far, lies more clearly in reconciling internal state with recent evidence at prediction time than in any superiority as a rule for learning parameters.

The next priorities are replication on fresh corpora, longer streams testing retention and plasticity under a fixed memory budget, stronger base models under the caps, and direct measurement of how well the cost of structural search is amortized. A parallel symbolic track should test typed program readers and held-out compositions. Decentralized exchange of modules, and bases prepared jointly with their caps, are natural later extensions. The intended outcome is a learner that can preserve, access, revise and share useful structure throughout its operation. The results reported here establish some first mechanisms toward that outcome, and the greater part of the program is still ahead.

A Geometric preservation and the composition of errors

A.1 Derivation of the local predictive metric

This subsection derives the quadratic expansion stated after Equation (11). For a fixed continuation c , write $p_y = p_c(y \mid z)$ and expand

$$\text{KL}(p(z) \parallel p(z + \Delta)) = \sum_y p_y [\log p_y - \log p_c(y \mid z + \Delta)].$$

The term linear in Δ vanishes because $\sum_y \nabla p_y = 0$. The quadratic term is

$$\frac{1}{2} \Delta^\top \left(\sum_y \frac{\nabla p_y \nabla p_y^\top}{p_y} - \sum_y \nabla^2 p_y \right) \Delta,$$

and the second sum is zero, again because the probabilities sum to one. Averaging over continuations with the fixed weights ρ_c gives Equation (11). Strictly positive probabilities and local smoothness are essential to the argument. A distribution whose support changes, or a program with discontinuous branches, needs a different treatment.

A.2 Preserving a composition of stages

Suppose a pipeline has stages $i = 1, \dots, n$. Stage i has an ideal map F_i and an approximation $\hat{F}_i$ between metric spaces, and the approximation errs by at most ϵ_i uniformly. Assume that F_i is L_i -Lipschitz on all relevant states, meaning that it magnifies distances by at most the factor L_i . Let e_i be the discrepancy between the ideal and the approximate pipelines after stage i . Then

$$e_i \leq L_i e_{i-1} + \epsilon_i, \quad \text{hence} \quad e_n \leq \sum_{i=1}^n \epsilon_i \prod_{j=i+1}^n L_j$$

when both pipelines start from the same state. The first inequality follows by inserting $F_i(\hat{z}_{i-1})$ between the two outputs and applying the triangle inequality. The consequence is that if a later stage magnifies small errors, preserving behavior locally at each stage is not enough unless that magnification is controlled.

Exact quotients compose more simply. If every downstream map factors through each earlier quotient, the composed output is unchanged. For this to hold, the family of continuations has to include the relevant downstream computations. Checking an isolated, immediate readout is not sufficient.

The same reasoning applies to the order of updates. For update maps U and V , suppose $d(UV(z), VU(z)) \leq \eta$ on the relevant domain. Then a later L -Lipschitz continuation sees a discrepancy between the two orders of at most $L\eta$. This is a usable contract of near-commutation. It does not follow merely from using PC, or from storing both modules, and repeated swaps accumulate error, each with its own downstream amplification.

B Exact bounds for finite compression decisions

Section 4.6 distinguished predictions made by the search guide from verified facts. This appendix shows what verified facts can deliver: guaranteed intervals for the cost of a candidate codec, computable by linear programming. The guarantees hold for the specified finite model, abstraction of the data and objective. They do not solve unrestricted program search or nonconvex neural optimization.

B.1 A compatible histogram and a cost table

Let Ω be a finite alphabet of possible observation rows. A fixed dataset of N rows has an empirical distribution p over Ω . We may not know p exactly, but verified counts or intervals confine it to a nonempty polytope

$$\mathcal{P} = \{p \geq 0 : \mathbf{1}^\top p = 1, Ap = b, Bp \leq h\}. \quad (40)$$

A candidate executable codec θ has a fixed header cost H_θ , a code length $\ell_\theta(x)$ and an execution cost $t_\theta(x)$ for each row x , and nonnegative discrepancies $d_{\theta,c}(x)$ on each row for finitely many behavioral probes c . Its total cost is

$$J_\theta(p) = H_\theta + v_\theta^\top p + \mu \max_c d_{\theta,c}^\top p, \quad v_\theta = N\ell_\theta + \lambda t_\theta, \quad \mu \geq 0. \quad (41)$$

The final term is the worst *mean* discrepancy over the specified probes, which is a weaker thing than the worst error on any input. If ℓ is meant as a physical code length, the codebook must be valid, for example by satisfying Kraft's inequality. Arbitrary fitted scores are not code lengths.

Theorem B.1 (Finite fixed-candidate bounds). *Define*

$$\begin{aligned} L_\theta &= H_\theta + \min_{p,z} \{v_\theta^\top p + \mu z : p \in \mathcal{P}, z \geq d_{\theta,c}^\top p \ \forall c\}, \\ U_\theta &= H_\theta + \max_c \max_{p \in \mathcal{P}} (v_\theta + \mu d_{\theta,c})^\top p. \end{aligned} \quad (42)$$

Then $L_\theta \leq J_\theta(p) \leq U_\theta$ for every compatible actual histogram. Both endpoints are executable by finitely many linear programs. If the incumbent cost is J_0 , its gain lies in $[J_0 - U_\theta, J_0 - L_\theta]$.

Proof. The auxiliary z represents the finite maximum in the objective. For the upper endpoint, maxima over p and c commute. The actual histogram belongs to the feasible set, so its cost lies between the extrema. Subtraction reverses endpoints. $\square$

If the incumbent's cost is itself known only to lie in $[L_0, U_0]$, a safe interval for the gain is $[L_0 - U_\theta, U_0 - L_\theta]$. Bounding the paired difference in cost directly over the same compatible set can be tighter, because uncertainties common to both cancel.

For a finite family Θ_a of refits, the best cost $J_a^*(p) = \min_{\theta \in \Theta_a} J_\theta(p)$ satisfies

$$\min_\theta L_\theta \leq J_a^*(p) \leq \min_\theta U_\theta.$$

When the costs are affine in p , the tighter robust upper endpoint $\max_{p \in \mathcal{P}} \min_\theta J_\theta(p)$ is itself a linear program: maximize z subject to $z \leq J_\theta(p)$ for every candidate. It is at most $\min_\theta \max_p J_\theta(p)$, and no equality between the two is assumed.

One caution is essential. An upper bound on the cost of the *optimal* refit is not a guarantee about what an arbitrary heuristic refitter will achieve. A candidate should be accepted on the strength of a bound computed for the witness that was constructed, and that is feasible for the preservation requirements, or else with a justified allowance for the optimization gap. A score predicted by the guide is neither kind of certificate.

B.2 Checking numerical LP output conservatively

Numerical LP solvers return approximate answers, but a sound bound can still be extracted from them. For rational multipliers y, z , put $r = c - A^\top y - B^\top z$. Then

$$\begin{aligned} z \leq 0 &\implies \min_{p \in \mathcal{P}} c^\top p \geq b^\top y + h^\top z + \min_x r_x, \\ z \geq 0 &\implies \max_{p \in \mathcal{P}} c^\top p \leq b^\top y + h^\top z + \max_x r_x. \end{aligned} \quad (43)$$

To prove these, expand $c^\top p = b^\top y + z^\top Bp + r^\top p$, use the sign of the multiplier, and bound the last term, which is a probability-weighted average, by the extreme values of the residual. The approximate multipliers returned by a solver can therefore be rounded to explicit rationals and used with a conservative residual correction. Finding valid global extrema of the residual can itself be hard, and a heuristic extremum that errs in the wrong direction is not sound.

As new sound observations shrink $\mathcal{P}$, the exact lower extrema rise and the upper extrema fall, provided the candidates and the objective stay fixed. If the data, the code table, the library or the objective changes, a new bounding episode has to be started, or the constraints have to be carried over explicitly. Preservation requirements can be checked by maximizing their discrepancies over $\mathcal{P}$. A candidate that has not been certified feasible cannot be accepted just because the bound on its unconstrained cost looks good.

B.3 Sparse dependencies and a bound on the remainder

For binary variables, any function of a row can be written in the Walsh basis,

$$f(x) = \sum_{S \subseteq [d]} \hat{f}(S) \chi_S(x), \quad \chi_S(x) = \prod_{i \in S} (-1)^{x_i},$$

in which the term for a subset S captures the interaction among exactly the variables in S . Since $|\mathbb{E}_p \chi_S| \leq 1$, discarding all terms above order k changes the expectation by at most

$$\left| \mathbb{E}_p f - \sum_{|S| \leq k} \hat{f}(S) \mathbb{E}_p \chi_S \right| \leq \sum_{|S| > k} |\hat{f}(S)|. \quad (44)$$

The bound is useful only when the tail of coefficients is known, or can be bounded from explicit structure. Low-order observations alone do not imply a small tail, and parity is the counterexample. Relaxations to local marginals are another option. Every global histogram induces consistent tables on regions, so minimizing over the larger set of locally consistent tables gives a lower bound. Local consistency around a cycle does not suffice for global realizability. The cost of exact inference is governed by the treewidth and the order of contraction, and not just by the size of individual factors.

B.4 Partial evaluation and sampled evaluation

If every loss lies in $[0, B]$, then after evaluating m of N rows with running total S_m , the full mean is bounded by

$$\frac{S_m}{N} \leq \bar{L} \leq \frac{S_m + (N - m)B}{N}.$$

H2's mixtures have a uniform floor of $1/65,536$, so $B = 16$ bpb is a valid bound for them. These deterministic bounds can be loose, but they require no assumption of independence.

A different guarantee comes from sampling. Fix M candidate predictors *before* drawing n independent uniform samples, with replacement, from a fixed finite evaluation corpus. If each paired gain lies in $[a, b]$, Hoeffding’s inequality with a union bound gives simultaneous intervals for the mean gains,

$$\bar{g}_j \pm (b - a) \sqrt{\frac{\ln(2M/\delta)}{2n}}, \quad (45)$$

valid with probability at least $1 - \delta$. The statement concerns means over the fixed corpus, and it holds even if adjacent rows of the corpus are dependent, because the sampling draws are independent given the corpus. It does not make contiguous future text independent and identically distributed. If candidates are constructed adaptively, or stopping is repeated, either fresh evaluations or a valid sequential or multiple-testing construction is required. The descriptive block intervals reported for the experiments should not be relabeled as certificates of this kind.

B.5 A useful convex subproblem: mixture weights

For a fixed set of positive component probabilities $p_j(y_i | x_i)$, define

$$L_A(w) = -\frac{1}{N} \sum_i \log_2 \left(\sum_j w_j p_j(y_i | x_i) \right), \quad w \in \Delta.$$

This objective is convex in w , which allows the quality of any proposed weights to be certified. If $g = \nabla L_A(w)$, then

$$0 \leq L_A(w) - L_A(w^*) \leq \langle g, w \rangle - \min_j g_j. \quad (46)$$

Convexity gives $L_A(w^*) \geq L_A(w) + \langle g, w^* - w \rangle$, and minimizing the linear form over the simplex gives the bound. What is certified is the numerical optimization of the weights for a fixed pool. The discrete search for components, the neural gate and generalization are not covered. In the v6 experiments the recorded median and maximum gaps on calibration data were 6.17×10^{-7} and 5.79×10^{-6} bpb.

C A finite compression example with exact arithmetic

This example makes concrete the claim of Section 4.6 that one high-order observation can settle a compression decision. Let $N = 1024$ observations each consist of three bits (A, B, C) . Each of the four even-parity symbols appears 254 times and each of the four odd-parity symbols appears twice, so there are eight exceptions to the rule $C = A \oplus B$. Every pairwise marginal is exactly uniform.

A raw code uses $3N = 3072$ bits. Now consider a fixed library convention under which invoking the parity feature costs a 24-bit header. The code stores the A and B streams, then the number of exceptions m in $\lceil \log_2(N + 1) \rceil$ bits, and then the rank of the set of exceptions among all $\binom{N}{m}$ subsets of that size. The resulting length in bits is

$$L(m) = 24 + 2N + \lceil \log_2(N + 1) \rceil + \left\lceil \log_2 \binom{N}{m} \right\rceil. \quad (47)$$

Once the decoder has read m it knows the length of the rank field, and it reconstructs C from the parity rule and the mask of exceptions. Framing shared by both alternatives is omitted from both. The 24-bit header is a convention adopted for this example and is not a universal cost of parity.

The extreme values in Table 11 follow from the symmetry and unimodality of $\binom{N}{m}$ in m : on an interval of integers its minimum is at an endpoint and its maximum is at a point closest to $N/2$. With only

Available evidence	Code-length interval	Gain over 3072 bits
Pairwise marginals only	[2083, 3102]	[−30, 989]
Certified $m \leq 16$	[2083, 2199]	[873, 989]
Exact $m = 8$	[2148, 2148]	[924, 924]

Table 11: One higher-order observation can settle a compression decision. The intervals are safe, even though additional integrality constraints might tighten them.

pairwise evidence the rate of exceptions is undetermined, and the parity feature could either save nearly a thousand bits or lose thirty. A measured bound on the number of exceptions makes the feature certainly worthwhile. Historical experience can suggest asking for that measurement early. It cannot take the place of making the measurement or proving the bound.

The example also separates two ideas that are easily run together. The fast predictor $\hat{C} = A \oplus B$ still makes m/N errors, while the decoder that knows the residual reconstructs the data with no error at all. Approximate prediction and exact preservation of information are different things, and a short high-order program can be worth searching for on either count.

D Implementation and data details

D.1 The H2 reader grammar and fitting configuration

The 13 readouts are:

1. The frozen transformer distribution.
2. Suffix depths 1, 2, 3, 4, 6, 8 with backoff strength $\alpha = 4$.
3. Suffix depths 3, 6 with backoff strength $\alpha = 16$.
4. Copy matches of suffix lengths 1, 2, 3 with fallback weight 2 to the base.
5. Copy length 3 with fallback weight 2 to the suffix-depth-6, $\alpha = 4$ reader.

Every distribution includes a uniform smoothing floor with $\epsilon = 1/256$, applied as $p \mapsto (1 - \epsilon)p + \epsilon/256$, and the copy mixtures preserve the same floor. The dictionary retains contexts with support of at least two, up to 16,000 per depth for depths 1–8, chosen by support with deterministic tie-breaking. Backoff from the shortest contexts starts from a smoothed unigram distribution.

The gate’s four features per reader are the entropy divided by $\log 256$, the peak probability, $\log(1 + \text{support})/10$, and the probability the reader assigns to the base’s most likely byte. Means and standard deviations from the fitting set are used for normalization, with standard deviations floored at 0.05 and normalized coordinates clipped to $[-8, 8]$. The feature coordinates of inactive readers are masked. Temporary activities are reinitialized for every prefix, so no inference state persists across evaluated examples.

The predefined five-reader graph contains the base, suffix depths 2, 4 and 6 at strength 4, and copy length 2. The graph transferred from development contains the base, suffix depth 2 at strength 4, the copy/suffix reader of length 3, suffix depth 6 at strength 16, and copy length 2. The five compressed PC graphs vary by seed. Most retain the base, suffix depth 8 at strength 4, suffix depth 3 at strength 16, copy length 2 and the copy/suffix reader of length 3. Seed 19 uses copy length 3, and seed 71 includes suffix depth 2 at strength 4 in place of suffix depth 3 at strength 16.

The raw Tiny Shakespeare file has 1,115,394 bytes. Its boundaries for training, validation and testing

are at bytes 0, 1,003,854, 1,059,624 and 1,115,394. Dictionary fitting ends at byte 700,000. Positions for fitting the gate are sampled from the training region after that point, with a margin of 64 bytes for context. Configuration tuning uses 4,096 targets from the first half of the validation region, and the selection of checkpoints and graphs uses 4,096 from the second half. The base model, which was inherited from earlier experiments, had already used the training portion for its own fitting and the validation portion for its own selection.

The two final test segments contain relative positions 64–32,831 and 32,832–55,769 of the test region. The five compression seeds are 7, 19, 37, 53 and 71. The final choices of architecture and checkpoint were locked before these H2 scores were evaluated. The reuse of one corpus across the whole research sequence remains a limitation despite this separation within the H2 runs.

D.2 Forgetting data and replay

The Alice text is Project Gutenberg ebook 11 [24]. Boilerplate was removed and typographic punctuation was normalized to ASCII. The processed file has 144,862 bytes, with split boundaries at 0, 101,403, 123,132 and 144,862. Each region supplies only those targets whose complete 64-byte prefix lies inside the region.

Python training uses `argparse.py`, `ast.py`, `configparser.py`, `csv.py`, `enum.py`, `fnmatch.py`, `functools.py`, `heapq.py` and `pickle.py`, 452,358 bytes in all. Validation uses `difflib.py`, 83,368 bytes. Testing uses `fractions.py`, `pathlib.py`, `random.py`, `statistics.py`, `textwrap.py` and `tokenize.py`, 215,403 bytes. The exact Python 3.12 sources, their license and their hashes are kept in the experiment archive.

The replay buffer consists of 1,024 examples from the original gate-training data, sampled separately for each seed. Cached features and reader likelihoods remain valid for replay because the base, the dictionary and the normalization are all frozen. If any of those changed, the cache would have to be recomputed, or raw contexts replayed instead. Held-out targets are never replayed. The old-task test segments were scored only after all 35 runs had finished and their checkpoints had been locked.

E Additional numerical tables

E.1 The full H2 comparison

The times in Table 12 are approximate CPU times for fitting the gate and searching. They exclude the construction of readers, the generation of base and reader caches, and final evaluation, and some independent jobs overlapped. An arm with all 13 readers uses 672 ordinary fitting updates, whereas the 1,056 updates of the compression arm include rejected candidates and controls. The reference code keeps the common dictionary and the full gate allocation after pruning, so the number of active readers is not a measured end-to-end gain in storage or speed.

E.2 The earlier v6 evaluation segment

Table 13 gives the v6 scores on the 16,384-byte continuation that had been evaluated previously, excluding the original familiar 4,096 bytes. This is not the 32,768-target segment used for H2. The saved v5 branch had a different budget for search and selection. The differences between the two v6 segments show why a single aggregate ranking of versions would mislead.

Condition	Original	Following	Readers	Updates	Seconds
Fixed five, BP	2.356488	2.422106	5.0	240	4.26
Fixed five, PC	2.356485	2.421712	5.0	240	4.35
All 13, BP	2.332462	2.403008	13.0	672	14.24
All 13, PC	2.332323	2.403255	13.0	672	13.96
Forward CD-BP	2.352200	2.423689	5.0	672	12.02
Forward CD-PC	2.352245	2.423750	5.0	672	11.75
Forward CD-PC, random	2.362251	2.431396	5.0	672	11.67
Reused graph, BP	2.341418	2.414357	5.0	240	4.32
Reused graph, PC	2.341418	2.414385	5.0	240	5.25
Reused graph, no inference	2.343121	2.415047	5.0	240	0.81
Compression-first CD-BP	2.334918	2.409146	5.0	1056	23.15
Compression-first CD-PC	2.335741	2.410003	5.0	1056	24.46
Fixed five, BP, matched	2.345939	2.415981	5.0	1056	15.99
Fixed five, PC, matched	2.345958	2.415914	5.0	1056	16.92
Reused graph, PC, matched	2.337033	2.410459	5.0	1056	17.25

Table 12: All 15 final H2 conditions, five-seed means. Scores are in bpb, and times cover gate fitting and search only.

Variant	Bigram	N-gram	Transformer
Frozen base	3.539136	2.343373	2.874901
Saved v5 individual	3.114650	2.339908	2.823919
Coarse proxy	3.091512	2.339980	2.832276
Aligned proxy	3.049378	2.341292	2.838522
Independent simple history	3.044908	2.336760	2.837313
Trajectory simple history	3.043991	2.338553	2.844794
Trajectory rich history	3.042865	2.337775	2.844293
More precise outcome labels	3.042695	2.336942	2.834262
Online guide updates	3.045572	2.337628	2.840607
Compact shadow trials	3.062609	2.335110	2.833682

Table 13: CD text v6 on the earlier 16,384-byte continuation, two-seed means.

Domain	Update	Seed range of F_A	Block interval	Following F_A
Alice	PC	[0.05877, 0.07278]	[0.05818, 0.07335]	0.074235
Alice	BP	[0.05923, 0.07296]	[0.05838, 0.07354]	0.074636
Alice	PC + replay	[0.03114, 0.04398]	[0.03422, 0.04343]	0.043715
Python	PC	[0.07497, 0.08454]	[0.07299, 0.08509]	0.080161
Python	BP	[0.07544, 0.08488]	[0.07355, 0.08567]	0.080693
Python	PC + replay	[0.04133, 0.05583]	[0.04450, 0.05425]	0.051220

Table 14: Consistency of forgetting at the fixed-budget endpoint, with descriptive 95% paired block-bootstrap intervals.

Domain	Update	Selected updates by seed	F_A	G_B
Alice	PC	640, 640, 960, 640, 1024	0.059998	0.087389
Alice	BP	640, 640, 960, 640, 1024	0.060076	0.087372
Alice	PC + replay	640, 640, 960, 768, 960	0.034942	0.085882
Python	PC	1024, 768, 1024, 1024, 640	0.076247	0.251383
Python	BP	1024, 768, 1024, 1024, 640	0.076797	0.251352
Python	PC + replay	1024, 768, 704, 960, 960	0.049853	0.249227

Table 15: Endpoints selected using new-domain validation alone. The selection made no use of retention scores on the old domain.

E.3 Forgetting ranges and validation-selected endpoints

The intervals in Table 14 resample consecutive blocks of 256 held-out targets, after the paired differences have been averaged across seeds. They describe variation within the measured Shakespeare stream. They do not describe variation across independent corpora or independently pretrained bases. All final PC forgetting values are positive, and some forgetting remains in each replay arm.

In Table 15 the selected update counts are listed for seeds 7, 19, 37, 53 and 71 in that order. Selection uses only new-domain validation, includes step zero as a candidate, and never stops the scheduled fixed-budget run early. The primary endpoints remain the values at 1,024 updates given in Section 9.

F Provenance of the evidence, verification and reproduction

F.1 Where the numbers come from

No language model was trained in the course of writing this report. It brings together the theory with experiments that had already been run and archived, and every number in it comes from the project’s experiment records. The main tables for H1, H2, v6 and forgetting are generated from machine-readable analysis files kept with the source of this document. The numbers for the finite tasks and for CD text v1–v5 are transcribed from the project’s methodology report, and those experiments were not rerun.

The source bundle contains the complete ASCII LaTeX with embedded diagrams, plots and bibliography, the code that generates the tables, a compact script that checks the numbers, and the JSON and Markdown analysis files behind the principal tables. It does not duplicate all the model checkpoints and corpora in the experiment archives. The LaTeX file has its tables and figure coordinates embedded, so it compiles independently wherever the required TeX packages are installed. The standard `algorithmicx` and `algpseudocode` package files are included for convenience.

Evidence	Primary file or archive	What it substantiates
CD foundations	<code>compression_distinction_networks.tex</code>	Definitions, finite bounds, parity example, cap design
Earlier CD record	<code>cd_methodology_and_sandbox_results.pdf</code> , revised	Synthetic and text v1–v6 narrative; earlier numerical tables
CD v6	<code>cd_text_v6/RESULTS_V6.md</code> ; pilot <code>summary.json</code>	48 runs, history audits, both evaluation segments
Hybrid H1	<code>cd_pc_hybrid/RESULTS.md</code> ; pilot <code>analysis.json</code>	24 main runs and auxiliary audits
Hybrid H2	<code>pc_cd_v2_sandbox.zip</code> ; <code>results/analysis.json</code>	75 locked models, 450 loss means, seven tests
Forgetting	<code>pc_cd_forgetting_experiment.zip</code>	35 adaptations, 595 checkpoints, 1,126 loss means

Table 16: Provenance map. The bundle identifies the exact evidence files copied into it, with their hashes. These are internal records and are not claimed to be peer-reviewed publications.

Table 16 maps each body of evidence to its archive. Readers who consult the archives will meet some different names. There, H2 appears as “PC–CD v2” and H1 as the “CD/PC hybrid.” The present names were chosen to prevent confusion between H2 and the CD text prototype v2. Some archived records also give the base transformer 136,960 parameters, a figure that counts the tied embedding and output matrix twice. The count of distinct parameters is 120,576, as stated in Section 7. The fuller internal accounts of the theory, the geometric formulation, the methodology and the individual experiments are [18, 20, 19, 21, 22, 23].

F.2 The scope of the recorded validation

The finite prototype reported 18 tests and exact round trips of code packets. CD v6 reported 14 implementation tests, 12 round trips through an arithmetic coder using real models, 6,291,456 causal checks of prefixes and branches, and 10,734 matched-root records. These counts document the implementation work that was recorded. They are not independent statistical replications.

H1 reported nine checks. They covered the derivatives of the energy, convergence of the weak nudge, descent during settling, growth that preserves the computed function and the continuity of the optimizer, causal prediction and readers, the algebra of screening, soft targets, and bitwise-identical continuation. H2 reported seven mathematical, causal and checkpoint checks, an audit of gradients and settling in the trained state, and the recomputation of 450 means.

The forgetting experiment added three checks: that the data roles are disjoint, that replay is drawn only from old training targets, and that training which is interrupted and resumed is bitwise equivalent to uninterrupted training. Its audit recomputed 1,126 loss means and verified all 595 checkpoint hashes, the initial gates, the frozen base, the dictionary and the fingerprints of the training sources. During an earlier audit, some H2 diagnostic loss files had to be regenerated by running inference alone from a locked checkpoint. No model was refitted, the previously recorded means were recovered, and the original archive records the repair.

F.3 A suggested sequence for replication

1. Recompute the tables from the bundled JSON and check the illustrative coding arithmetic. Compile this document and confirm that the generated table values agree.

2. Use the H2 and forgetting archives to rebuild the omitted feature caches, and rerun their locked recipes in a fresh results directory, keeping the original evidence separate.
3. Repeat the fixed recipe on fresh corpora before any further tuning. Include a strong base, a fixed cap, the full reader bank, CD-BP, CD-PC, and controls without settling and with replay or retained branches.
4. For studies of the guide, split the data by search trajectory as well as by text, keep the negative outcomes, and separate the calibration of the guide from the final audits. Account for all warm-up and search costs.
5. Increase scale only after independent gains survive matching of compute and memory. Treat the goals of Table 1 as separate measurements. They should not be merged into a single criterion of success.

G Glossary

Term	Meaning in this report
Observer	A function that returns a specified fact, feature or predictive distribution from a state.
Distinction	A difference between states that an observer can detect.
Pseudometric	A distance-like function that can assign zero distance to internally different states.
Coupling	Input-dependent routing weights from a destination to its allowed sources; each row sums to one.
Tagged measure	A representation of selected payloads with their weights and source identities, before aggregation.
Lift	The addition of derived coordinates while the original state is retained.
Continuation	A declared later use of a representation, including the relevant downstream computation.
Behavioral quotient	The summary obtained by identifying states that all specified continuations treat identically.
Fiber	The set of states that a re-encoding sends to the same code.
GT	Generalized transformer: a layer organized as routing, transport, lifting, optional reconciliation and re-encoding.
CD	Compression-driven distinction architecture: executable structure, residuals, refitting and structural search.
RBII	Resource-bounded incremental induction: bounded program discovery, reuse, and online prediction and adaptation.
MDL	Minimum description length: prefer the explanation that minimizes the total length of model plus data.
Dependency tower	Nested families of observations of selected interactions, with explicit order and resource budgets.
STLM-style guide	A scorer, trained on search history, for logical or program templates, their contexts and search actions.
Proxy	A cheap ranking of candidates from the current residual, with no learned history.

Term	Meaning in this report
PC	Predictive coding; here chiefly learning from free and nudged activity states, and explicit reconciliation of state with observed evidence.
BP	Backpropagation through the declared computation, including unrolled inference when present.
Settling	Iterative adjustment of temporary activities to lower an energy.
Cap	A learned attachment that reads a base model's outputs or states and supplies an additional prediction or correction.
Reader	A component of a cap that proposes a next-token distribution from a memory, such as a suffix or copy model.
Gate	The small trainable part of H2 that sets the trust weights over readers.
Shadow	A compact alternative trained from a retained teacher, available as another predictor and as a starting point for later search.
Replay	Mixing examples from earlier training data into the batches used for adaptation.
bpb	Bits per byte: the average negative base-two logarithm of the probability given to the observed next byte.
Forgetting	The increase in old-domain loss after learning a new domain, measured together with new-domain learning.

References

- [1] A. Vaswani et al. *Attention Is All You Need*. 2017. <https://arxiv.org/abs/1706.03762>.
- [2] J. Kirkpatrick et al. *Overcoming Catastrophic Forgetting in Neural Networks*. 2017. <https://arxiv.org/abs/1612.00796>.
- [3] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt. *Test-Time Training with Self-Supervision for Generalization under Distribution Shifts*. ICML 2020. <https://arxiv.org/abs/1909.13231>.
- [4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. AISTATS 2017. <https://arxiv.org/abs/1602.05629>.
- [5] B. M. Lake and M. Baroni. *Generalization without Systematicity: On the Compositional Skills of Sequence-to-Sequence Recurrent Networks*. 2018. <https://arxiv.org/abs/1711.00350>.
- [6] J. Vuckovic, A. Baratin, and R. Tachet des Combes. *A Mathematical Theory of Attention*. 2020. <https://arxiv.org/abs/2007.02876>.
- [7] N. Tishby, F. C. Pereira, and W. Bialek. *The Information Bottleneck Method*. 1999/2000. <https://arxiv.org/abs/physics/0004057>.
- [8] B. Scellier and Y. Bengio. *Equilibrium Propagation: Bridging the Gap between Energy-Based Models and Backpropagation*. 2017. <https://arxiv.org/abs/1602.05179>.
- [9] B. Millidge, A. Tschantz, and C. L. Buckley. *Predictive Coding Approximates Backprop along Arbitrary Computation Graphs*. 2020. <https://arxiv.org/abs/2006.04182>.

- [10] G. Hinton, O. Vinyals, and J. Dean. *Distilling the Knowledge in a Neural Network*. 2015. <https://arxiv.org/abs/1503.02531>.
- [11] E. J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. <https://arxiv.org/abs/2106.09685>.
- [12] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis. *Generalization through Memorization: Nearest Neighbor Language Models*. 2020. <https://arxiv.org/abs/1911.00172>.
- [13] E. Grave, A. Joulin, and N. Usunier. *Improving Neural Language Models with a Continuous Cache*. 2017. <https://arxiv.org/abs/1612.04426>.
- [14] W. Fedus, B. Zoph, and N. Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. 2021/2022. <https://arxiv.org/abs/2101.03961>.
- [15] K. Ellis et al. *DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning*. 2020/2021. <https://arxiv.org/abs/2006.08381>.
- [16] B. Goertzel. *Incremental Compression: Theoretical Exploration of Multi-Feature Decomposition and Computational Limits*. Supplied manuscript, December 31, 2025. [incremental_compression_fixed_v3.pdf](#).
- [17] B. Goertzel, with GPT5-Pro assistance. *RBII + Causal Coding: Real-Time Module Adaptation with Near-Commuting Context Updates*. Supplied manuscript, February 15, 2026; discusses D. MacDonald’s RBII framework. [rbii_causal_coding_revised.pdf](#).
- [18] B. Goertzel, developed with ChatGPT. *Compression-Driven Distinction Networks: A Transformer Analogue with Incremental Compression, History-Guided Search, and Composable Caps*. Internal research synthesis, September 2026. [compression_distinction_networks.tex](#).
- [19] *Compression-Driven Distinction Networks: A Self-Contained Account of the Architecture, Learning Method, and Preliminary Experiments*. Research synthesis prepared for B. Goertzel, revised September 18, 2026. [cd_methodology_and_sandbox_results.pdf](#).
- [20] *A Geometric Framework for Transformers and Compression-Driven Networks*. Research synthesis prepared for B. Goertzel and colleagues, September 18, 2026. [geometric_cd_transformers_pc.tex](#).
- [21] *CD / Predictive-Coding Hybrid: Sandbox Results*. Internal implementation report and archived analysis, September 2026. [cd_pc_hybrid/RESULTS.md](#).
- [22] *PC-CD v2: Sandbox Results*. Internal implementation report and archived analysis, September 2026. [pc_cd_v2_results.md](#); code/evidence archive [pc_cd_v2_sandbox.zip](#).
- [23] *PC-CD v2: A Sequential-Learning Forgetting Test*. Internal implementation report and archived analysis, September 2026. [pc_cd_forgetting_results.md](#); [pc_cd_forgetting_experiment.zip](#).
- [24] L. Carroll. *Alice’s Adventures in Wonderland*. Project Gutenberg ebook 11. <https://www.gutenberg.org/ebooks/11>. Original downloaded text and processing details are preserved in the forgetting archive.